



Mi-Memory: A Lifecycle Memory Framework for Personal AI

Darwin Agent Team

See [Contributions and Acknowledgments](#) section for a full author list.

Abstract

Personal AI is moving beyond chat-only interaction toward continuous services that span phones, cars, homes, wearables, cameras, and tools. In this setting, memory cannot remain a cache of prior conversations. It should serve as a continuity and governance substrate: preserving durable user state, grounding answers in multimodal and device evidence, supporting correction and forgetting, bounding policy evolution, and remaining deployable under latency, cost, privacy, and edge-cloud constraints. This technical report presents **Mi-Memory**, a lifecycle memory framework for Personal AI organized around four roles: **Structure**, **Expansion**, **Evolution**, and **Deployment**. A shared audit contract links these roles through four recurring artifact families: typed evidence payloads preserve source identity and provenance, diagnostic traces localize evidence loss across the serving pipeline, strategy artifacts make memory-policy changes explicit, and gate/rollback records bound accepted evolution. Mi-Memory instantiates the roles through MemStack, MemSense/MemFuse, D²ACCI/E²MEND, and LiteMem. In controlled-reference Structure evaluations, MemStack reaches 93.59%, 57.24%, and 87.47% on LoCoMo, PersonaMem-V2, and LongMemEval, respectively; other tracks report module-level, preliminary/internal, transfer-feasibility, or design-only evidence with explicit boundaries. Mi-Memory is a step toward auditable, evidence-gated, and deployment-aware memory systems for Personal AI. [Project homepage: https://darwin-agent.github.io/Mi-Memory/](https://darwin-agent.github.io/Mi-Memory/).

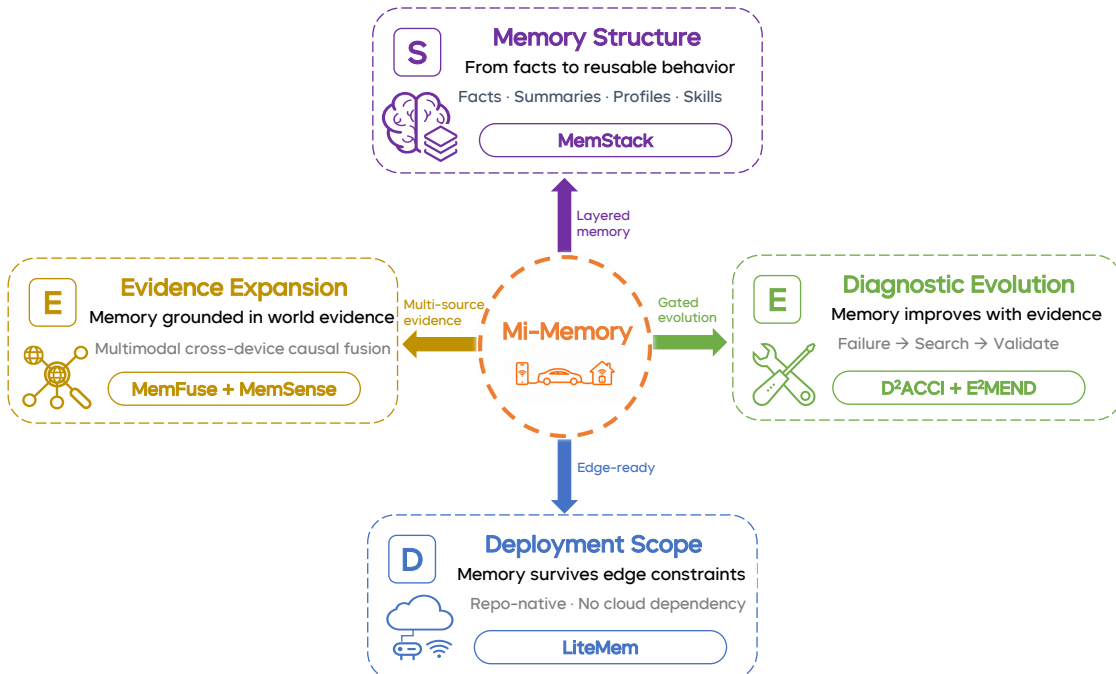


Figure 1 Mi-Memory lifecycle.

Contents

1	Introduction	3
1.1	Research Questions	3
1.2	Running Example: Human-Car-Home Training Handoff	4
2	Related Work	5
3	Problem Formulation and System Overview	7
3.1	Formulation: Lifecycle Audit Contract	7
3.2	Framework: Lifecycle Architecture and Interfaces	9
4	Structure: Structured Memory Substrate with MemStack	10
4.1	Design Motivation: Continuity as an Audit Obligation	10
4.2	Method: Trace-Producing Memory Runtime	11
4.3	Validation: Controlled Reference for Continuity	13
5	Expansion: Evidence Expansion with MemSense and MemFuse	13
5.1	Design Motivation: World Evidence Beyond Dialogue	14
5.2	Method: Evidence Admission Pipeline from Grounding to Fusion	15
5.3	Validation: Evidence Coverage in Realistic Daily Scenarios	17
6	Evolution: Governed Evolution from D²ACCI to E²MEND	19
6.1	Design Motivation: Manual Diagnostic Iteration Cannot Keep Pace	19
6.2	Method: Dual-Loop Framework for Bounded Evolution	20
6.3	Validation: Strategy Improvement under Bounded Automation	23
7	Deployment: Lightweight Memory with LiteMem	25
7.1	Design Motivation: Memory under Deployment Constraints	25
7.2	Method: Repository-Native Audit Substrate	25
7.3	Validation: Substrate Transfer	27
8	Evidence-Governed Claims Across the Lifecycle	27
8.1	Evidence-Governed Evaluation	28
8.2	Cross-Module Findings and Limitations	28
9	Conclusion and Outlook	29
	Contributions and Acknowledgments	34
	Appendix	35

1 Introduction

Personal AI is moving from a chat-only interface toward continuous, cross-device services. Its memory layer must maintain user state over time, connect evidence across phones, cars, homes, wearables, cameras, and tools, and revise or forget records as user intent and context change. This layer functions as both an architectural substrate and a governance surface: it determines what the assistant can know, explain, revise, and reuse under stated controls while making visible which observations are active, which sources are linked, and which updates are permitted.

These requirements make the systems problem broader than long-term recall. First, there is a **state gap**: durable facts, summaries, profiles, sessions, and procedures are often maintained at incompatible granularities. Second, there is a **source gap**: dialogue, images, tool traces, and device events are indexed by different subsystems, so cross-device evidence can disappear before it reaches generation. Third, there is a **governance gap**: final answer accuracy rarely reveals whether a failure began in ingestion, fusion, retrieval, filtering, context assembly, or generation, and unchecked policy updates can silently regress hidden slices. Finally, there is a **deployment gap**: memory design must account for cost, latency, privacy, and cloud/edge/local substrates instead of assuming a single vector-store runtime.

These gaps share a common cause: memory is treated as a static store, not as a *lifecycle*. In practice, assistant-relevant evidence passes through stage transitions—observation, structuring, fusion, retrieval, assembly, response, correction, and deployment—and each transition can lose, distort, or silently regress information. A camera frame may be captured but never linked to its causal context; a fused episode may exist in storage but fail retrieval filtering; a promising policy change may be deployed but silently degrade a hidden question category. The lifecycle perspective therefore shifts the target from optimizing one isolated memory store to defining an end-to-end contract that makes evidence movement traceable and stage transitions auditable.

Mi-Memory operationalizes this perspective through four lifecycle roles. **Structure** organizes admitted observations at the appropriate granularity and keeps them retrievable across time. **Expansion** admits new evidence sources—images, device events, cross-device links—without losing identity or provenance. **Evolution** ties each memory-system change to a falsifiable hypothesis, fixed evaluation conditions, and reversible update records. **Deployment** preserves audit obligations when memory moves across serving substrates—from cloud to edge to local files. Later sections ground these roles in concrete modules and explicit evidence levels.

For example, in a multi-device consumer ecosystem such as Human-Car-Home, a restaurant destination selected on the phone may influence car navigation; a wearable sleep signal may contextualize a family routine; and an in-car reminder before a child’s training session may depend on a home-camera observation, a calendar event, route timing, and prior user corrections, rather than any single chat turn. The lifecycle framing matters because this cross-device evidence must survive every stage transition from observation to response, and failures can surface at any point along the way.

1.1 Research Questions

Mi-Memory is organized around four research questions that turn the gaps above into the report structure. Table 1 is a roadmap, not a leaderboard: each row names the lifecycle role being tested, the artifact family that carries the audit obligation, the section where the mechanism is instantiated, and the evidence boundary for interpreting the claim. The table also separates two questions that are often conflated in memory-system papers: whether the runtime preserves task-relevant state, and whether later lifecycle functions—evidence admission, policy evolution, and deployment transfer—preserve the same traceability obligations.

Together, the rows emphasize lifecycle inspectability over any single end-to-end benchmark. RQ1 anchors the runtime substrate: the **Structure** role asks whether structured memory records, retrieval traces, and adapter contracts can support controlled memory QA (instantiated as MemStack, Section 4). RQ2 widens the evidence universe: the **Expansion** role tests whether visual and cross-device observations can enter the system as typed, provenance-preserving payloads (via MemSense and MemFuse, Section 5). RQ3 introduces governed change: the **Evolution** role asks whether diagnostic traces can become reversible strategy updates instead of silent prompt or retrieval edits (via D²ACCI and E²MEND, Section 6). RQ4 tests transfer: the

Table 1 Research-question map.

Research question	Lifecycle	Main artifact	Section	Evidence level
RQ1: How can long-term assistant memory preserve task-relevant user state without collapsing into unstructured chunks?	Structure	Layered memory records, retrieval traces, adapter contracts	Sec. 4	Controlled reference
RQ2: How can memory incorporate evidence from cameras, wearables, cars, homes, and other devices without losing provenance?	Expansion	Typed source observations, MemoryPacks, FusionSession graphs	Sec. 5	Module-level and preliminary/internal
RQ3: How can memory systems improve without silent regressions or hidden framework edits?	Evolution	D ² ACCI traces, E ² MEND strategy artifacts, gates, rollback records	Sec. 6	Controlled offline benchmark / descriptive
RQ4: How can the same memory contract transfer to privacy-sensitive or edge-oriented deployment substrates?	Deployment	Markdown/Git memory files, repository traces, file-tool retrieval	Sec. 7	Transfer-feasibility

Deployment role asks which parts of the same contract survive when the serving substrate becomes repository-native and lightweight (via LiteMem, Section 7).

The central systems question is whether these roles preserve the shared artifacts needed for bounded, auditable iteration across the memory lifecycle. This organization motivates the later pairing of accuracy or qualitative examples with the artifact that makes each claim inspectable: typed payloads, diagnostic traces, strategy artifacts, gate decisions, rollback records, or repository history.

1.2 Running Example: Human-Car-Home Training Handoff

The following Human-Car-Home trace makes the lifecycle concrete. On Tuesday afternoon, a parent is driving Ethan to basketball training. The car has left the residential community; the phone calendar records a 6:00 p.m. training session; the in-car navigation system estimates whether a short return home would still allow on-time arrival; and a home camera frame suggests that Ethan’s training bag may still be near the entrance. Before entering the expressway, the parent asks the in-car assistant: “Is there anything we should check before heading to training?”

A dialogue-only memory may have access only to the current question and return a general checklist. A lifecycle-aware memory system should instead preserve the evidence chain that makes a specific answer possible:

- **Observation:** the camera frame, calendar event, and vehicle route state are each captured as typed evidence with device identity and timestamp.
- **Structuring:** each observation is stored at an appropriate granularity—an atomic event, a session summary, or a stable profile entry—so it can be distinguished from other evidence over time.
- **Fusion:** related observations are linked into a coherent episode (training bag + scheduled practice + return-time estimate) despite arriving from different devices without shared keywords.
- **Retrieval and assembly:** when the in-car query arrives, the system recovers the fused episode and packs it into a bounded answer context alongside relevant prior corrections and family preferences.
- **Response:** the answer can cite the evidence it used—the bag observation, the time estimate, the correction about spare equipment—making its reasoning inspectable.
- **Diagnosis:** if the answer is wrong, the system can localize whether the failure came from missed observation, broken fusion, retrieval filtering, or generation error.
- **Governed update:** if the parent corrects the assistant (“Ethan keeps spare shoes at school; only remind me when the jersey is missing”), the correction enters memory through a gated policy change, not a silent edit.
- **Deployment transfer:** the same episode can be stored as local repository artifacts (Markdown files with Git provenance) when privacy or offline constraints require it, preserving inspectability under a lighter retrieval policy.

This is a lifecycle trace, not an end-to-end product claim. The requirement is to preserve evidence identity, provenance, temporal order, device constraints, and causal links across every stage transition, while keeping each transition auditable. Section 3 maps these stages to concrete modules; later technical sections (Sections 4–7) describe how each stage is implemented and where evidence can still be lost.

Contributions. The report’s primary contribution is the lifecycle audit contract itself. It unifies algorithmic, systems, and governance aspects of assistant memory by placing multi-granularity state organization, hybrid retrieval composition, evidence admission, and bounded policy evolution under one diagnostic framework. The concrete contributions are:

- **Lifecycle formulation.** It formulates Personal AI memory as a traceable lifecycle problem and makes typed evidence, stage-local diagnostics, strategy artifacts, and gate/rollback records the organizing contract.
- **Structured runtime.** It presents MemStack as a typed, multi-granularity reference runtime with hybrid retrieval, bounded context assembly, optional procedural hooks, and controlled-reference memory-QA evidence.
- **Evidence expansion.** It studies Evidence Admission through MemSense and MemFuse, demonstrating that visual and cross-device observations can become provenance-preserving payloads while marking MemSense results as module-level evidence and MemFuse results as preliminary/internal evidence.
- **Governed evolution.** It connects D²ACCI and E²MEND to the same audit contract, demonstrating that diagnostic traces, schema-bounded strategy search, gate decisions, and rollback records can make memory-policy improvement explicit and reversible.
- **Deployment transfer.** It presents LiteMem as a repository-native transfer path for selected Mi-Memory contract elements, demonstrating that Markdown/Git artifacts and file-tool retrieval can preserve provenance, editability, and auditability under lightweight deployment constraints.

Section 8 centralizes the evidence-level taxonomy, statistical qualifiers, and follow-up validation plan.

2 Related Work

Assistant memory research spans several lines of work that are often studied separately. The systems closest to Mi-Memory cover the lifecycle’s complementary slices: durable state, source grounding, policy evolution, and deployable substrates.

General-purpose assistant memory systems. Agent memory is often framed as long-term dialogue recall, but Personal AI requires a broader substrate: durable state, multimodal and device-grounded evidence, managed update and forgetting, and deployment under cost, latency, privacy, and edge–cloud constraints. Prior systems relevant to this broader substrate fall into several families. Fact- and profile-memory systems such as Mem0 [7, 31] emphasize extraction, update, and retrieval. Hierarchical and agent-runtime systems organize profiles, episodes, summaries, memory-operation policies, and tool-facing memory operations [22, 34, 38]; OS-inspired systems such as MemGPT manage long-lived agent state through virtual context and memory tiers [26].

Managed assistant products such as Apple Intelligence’s on-device context and Google Gemini’s memory features indicate demand for persistent personalization, although their internal provenance, diagnostic, and evolution mechanisms are rarely public. Cross-scenario memory providers and newer benchmarks underscore the challenge of remaining reliable across heterogeneous tasks [6, 36]. The unresolved systems problem is how to connect these memory families through a shared audit contract: what evidence is created, how it crosses module boundaries, where it can be lost, how updates are governed, and which deployment substrate can serve it within stated constraints. Mi-Memory treats this contract as the connective layer across memory stores, source modules, evolution controls, and deployment substrates.

Memory safety, privacy, and governance. Memory also expands the safety surface beyond standard prompt context: stored facts can become stale, sensitive, over-generalized, or retrieved in the wrong situation. Recent memory-security work studies lifecycle attacks and defenses [18], trustworthy retrieval gates [43], and stale or harmful memories that remain frequently accessed [4, 30]. Industry systems increasingly emphasize user-visible control, local context, and editability, but public documentation rarely exposes the diagnostic pipeline behind those controls. Mi-Memory focuses on the audit-enabling layer of memory governance: it makes provenance, update decisions, forget constraints, and strategy changes explicit, so product-level consent mechanisms, access-control policies, and privacy policies can operate on inspectable artifacts, not hidden retrieval state.

Memory evaluation and diagnostic iteration. Benchmarks such as LoCoMo [23], PersonaMem-V2 [15], and LongMemEval [35] evaluate whether a memory system can answer questions over long histories. Newer representation and benchmark settings emphasize abstraction-specificity balance, forgetting-aware evaluation, and long-horizon agent behavior [33, 36, 38]. Multimodal memory benchmarks such as M³Exam [14] and MemLens [28] further test whether visual observations and cross-modal context persist across sessions, while arena-style evaluations such as EvoArena [39] track evolution robustness in dynamic environments. These benchmarks are necessary, but accuracy alone rarely explains why an improvement happened or why a regression occurred. Evaluation-driven iteration work shows that seemingly improved prompts or pipeline changes can hurt hidden categories [8], and data-centric AI highlights the value of systematic evidence over ad hoc model changes [41]. D²ACCI builds on this direction by making each iteration falsifiable: a change must be tied to a hypothesis, per-question diagnostic traces, paired comparisons, and a category-level non-regression decision.

Self-evolving memory and agent optimization. Self-evolving memory systems aim to reduce manual tuning by searching architectures, prompts, or memory-operation policies. EvolveMem explores automated memory architecture search [20], while Memory-R1 and Memento represent RL-style or case-based learning of memory-using agent policies [40, 45]. MIA couples non-parametric trajectory memory with a parametric planner through test-time learning in deep research agents [27]. Broader agent memory work studies adaptive collaboration, dual-process cognitive memory, and memory skills for fine-grained update decisions [10, 13, 42]. These approaches motivate automated improvement, but broad end-to-end objectives can complicate production debugging because indexing, extraction, retrieval, and presentation may all change at once. E²MEND adopts a narrower and more auditable stance: it automates only schema-constrained strategy changes inside a locked framework, while preserving candidate traces, gate decisions, and rollback records for later inspection.

Beyond dialogue and deployable memory. Many memory benchmarks still center on dialogue or text histories, while deployed assistants increasingly observe visual scenes, device events, tool traces, and user routines. Multimodal memory agents with dual-layer hybrid memory [11] and lifelong multimodal memory [21] demonstrate that visual, auditory, and contextual observations can be organized as persistent agent knowledge. This direction echoes the memory-stream idea from generative-agent simulations, where observations, reflections, and plans persist beyond one prompt. Other work frames memory as externalized agent context or as a metabolic process of continual update and forgetting [24, 44]. Mi-Memory incorporates these directions by treating non-dialogue inputs as typed source observations whose identity, time, device provenance, and causal links must survive into downstream memory and retrieval.

A second axis is where memory is stored. Work on direct corpus interaction, repository- or file-native agent memory, version-controlled reasoning traces, and lightweight memory-augmented generation indicates that assistant memory can reside in file-native and versioned substrates alongside vector/database runtimes and conversational transcripts [2, 9, 17, 25, 29]. These systems vary in scope: some change the retrieval interface, some externalize agent state into files or repositories, and others reduce memory-augmented generation cost. LiteMem follows this deployment-oriented line by testing whether selected Mi-Memory principles can transfer to Markdown/Git memory under lightweight local constraints, while optional procedural hooks remain a design-only interface for user-specific interaction procedures inside MemStack.

System-level positioning. Table 2 maps related work to its *primary contribution* within the lifecycle framing. The column heading uses “primary contribution” because many systems span multiple concerns—Memora covers both state organization and evolution, EvolveMem bridges evolution and deployment—but each is listed under the slice it most advances. Mi-Memory connects these slices under one audit contract across the Personal AI memory lifecycle.

Table 2 Related-work positioning by primary lifecycle contribution.

Primary contribution	Representative prior work	Mi-Memory module
State organization	Mem0 / MemGPT / HMO / MemMachine / Memora*	MemStack
Evidence grounding	MemVerse / Mem-Gallery / M ³ Exam / M2A / MemLens	MemSense + MemFuse
Policy evolution	EvolveMem* / Memory-R1 / MemSkill / SSGM	D ² ACCI/ E ² MEND
Deployment substrate	LightMem / Git-of-Thoughts / Git Context Controller	LiteMem

*Also contributes to an adjacent slice (Memora: evolution; EvolveMem: deployment).

The remaining gap is therefore operational, not purely algorithmic: Personal AI memory needs a contract through which provenance, typed interfaces, update governance, and deployment budgets can survive module boundaries. The following sections instantiate that contract through MemStack, Evidence Admission, D²ACCI/E²MEND, and LiteMem, while Section 8 specifies each claim’s maturity and boundary.

3 Problem Formulation and System Overview

3.1 Formulation: Lifecycle Audit Contract

To make the lifecycle view operational, this section names the objects that must remain traceable as evidence moves between stages. At turn t , let q_t denote the user request, O_t the typed source-observation pool available at serving time, M_t the memory state before serving, E_{q_t} the selected evidence, C_{q_t} the bounded generation context, and a_t the generated answer.

A general assistant memory system is modeled as two coupled processes: a serving runtime that produces the current answer and an improvement loop that changes memory state or policy across turns. The memory policy governs ingestion, retrieval, filtering, context construction, and update.

During serving, the runtime selects evidence $E_{q_t} \subseteq M_t \cup O_t$, assembles C_{q_t} , and generates an answer a_t grounded in E_{q_t} . Across turns, the improvement loop writes the next memory state M_{t+1} and may revise the memory policy only through governed evolution.

Unlike conventional RAG, O_t is not a fixed text corpus. It may contain dialogue turns, device events, visual observations, user corrections, profile updates, and deployment-specific constraints. The memory policy therefore governs retrieval, admission, fusion, invalidation, and the substrate that carries evidence.

The audit contract treats answer accuracy as necessary but insufficient because a correct answer can still hide an unsupported memory write or policy change. It separates *serving correctness* (whether a single answer is grounded) from *update correctness* (whether a memory or policy change can be trusted). A valid memory system must satisfy three operational requirements:

1. **Evidence preservation:** every memory-dependent answer should retain traceable links from output context back to source observations or stored memory items.
2. **Stage-local diagnosis:** when an answer fails, the trace should identify whether the earliest loss occurred in ingestion, storage, retrieval, filtering, context packing, or generation.
3. **Auditable evolution:** a policy change should be tied to a hypothesis, evaluated under a fixed harness, compared against a baseline, and either accepted or rejected with preserved artifacts.

Operationally, an answer satisfies the audit contract only if a_t can be traced back through the assembled context C_{q_t} , selected evidence E_{q_t} , and source or memory identifiers in $M_t \cup O_t$. A policy update satisfies the contract only if it carries a versioned strategy diff, a fixed harness, a gate decision, and a rollback point.

With those records in place, violations become observable: missing source identity, unlocalized evidence loss, an unversioned policy change, or an accepted update without a restore record.

The formulation also separates three objects that are often conflated. A *memory item* is stored runtime content; *evidence* is the subset of source observations or memory items used to justify a specific answer or update; and a *strategy* is an explicit configuration that changes how evidence is selected or assembled. The distinction matters because storing the correct memory item is not enough: retrieval, filtering, or context construction can still drop the evidence before generation.

Together, these distinctions define the audit contract used throughout the report. At each module boundary, the contract is carried by four artifact families: **typed evidence** preserves source identity and provenance; **diagnostic traces** record where selected evidence moved or disappeared; **strategy artifacts** make memory-policy changes explicit and versioned; and **gate/rollback records** capture whether those changes can be accepted.

A lifecycle role denotes an obligation in the audit contract, not a claim about empirical maturity. Structure preserves typed memory items; Evidence Admission preserves source identity across heterogeneous observations; Evolution makes policy changes falsifiable and reversible; and Deployment tests how much of the same contract survives under different serving constraints.

Table 3 Audit artifacts.

Artifact	What it preserves	Primary roles	Failure made visible
Typed evidence payload	Source id, time, device, confidence, and provenance	Structure; Evidence Admission	Missing, stale, or mis-linked evidence
Diagnostic trace	Stage-local movement from ingestion to answer context	Structure; Evolution	Earliest loss in retrieval, filtering, packing, or generation
Strategy artifact	Allowed memory-policy mutation and versioned parameters	Evolution	Hidden prompt or configuration drift
Gate/rollback record	Accept/reject decision, comparison baseline, and restore point	Evolution; Deployment	Silent regression or unsafe strategy adoption

Table 3 turns the formulation into module responsibilities: MemStack anchors typed state, MemSense and MemFuse admit heterogeneous source evidence, D²ACCI and E²MEND govern strategy change, and LiteMem tests substrate transfer.

Section 8 later separates *evidence level* (harness maturity) from *statistical strength* (repeated runs, confidence intervals, or paired tests). That separation keeps this overview focused on the audit contract while preserving explicit result boundaries for the empirical sections.

Assumptions, boundaries, and terms. Mi-Memory focuses on memory-dependent assistant behavior within a trusted serving environment and explicit user-data governance. Its contribution here is auditable memory behavior: source observations, stored items, retrieved candidates, strategy changes, and rollback records remain visible to downstream policy and product controls. This boundary sets the evaluation scope: the report evaluates recall, grounding, bounded offline evolution, source expansion, and transfer feasibility, while production privacy enforcement, online abuse handling, and user-facing consent controls remain platform responsibilities that require additional validation. The detailed risk-gap summary is moved to Appendix B so the main formulation can stay focused on the lifecycle contract.

The terms used throughout the report are: a *memory item* is a typed stored unit (L0 fact, L1 summary, L2 profile, optional procedural hook, or source payload); *evidence* is the subset of items or source observations that justify a specific answer or update; a *strategy artifact* is a declarative, versioned configuration that E²MEND can mutate; and a *harness* is a fixed evaluation environment. Full definitions, including FusionSession, MemoryPack, diagnostic trace, and gate, are in Table 14 (Appendix B).

The operational boundary separates the audited improvement process D²ACCI from the automated strategy-search step E²MEND. D²ACCI is a human-guided engineering methodology: its primary unit is a falsifiable human hypothesis plus evidence artifacts. E²MEND is an automated strategy-search engine inside that

methodology: its primary unit is a candidate strategy artifact evaluated by a locked harness. Code, schema, benchmark, and acceptance-rule changes remain D²ACCI decisions, not autonomous E²MEND acceptances.

3.2 Framework: Lifecycle Architecture and Interfaces

The framework is contract-first: each Mi-Memory component is introduced by the lifecycle obligation it carries and by how it produces, consumes, or audits one of four shared artifacts: typed evidence payloads, diagnostic traces, strategy artifacts, and gate/rollback records. This contract separates the serving runtime, which handles source admission and memory processing, from the evolution governance plane, which diagnoses and updates those capabilities.

The four lifecycle roles defined in Section 1 map the contract obligations to concrete modules. MemStack maintains the runtime substrate, MemSense and MemFuse admit heterogeneous evidence, D²ACCI and E²MEND govern bounded change, and LiteMem exposes a transfer-feasibility substrate. Figure 2 summarizes this module map and the typed information flow between roles.

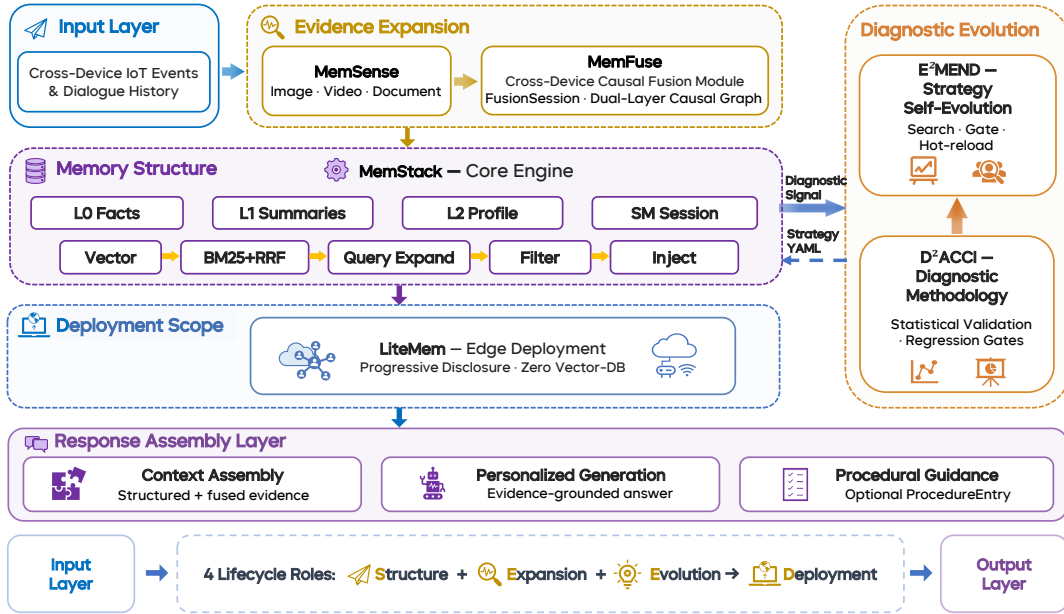


Figure 2 Mi-Memory flow.

Data-flow walkthrough. The running example in Section 1.2 is a module-interface walkthrough, not a second scenario. Source observations first enter the Expansion role: MemSense assigns stable identity to visual evidence, and MemFuse links related device events into MemoryPack-style payloads when causal evidence is sufficient. Those payloads then enter the Structure role through MemStack, where L0/L1/L2/SM records preserve atomic observations, session-level summaries, and stable family constraints.

At answer time, retrieval and context assembly select the relevant fused and structured evidence while preserving source identifiers in the assembled context. If the answer fails, the retained trace becomes an Evolution input for D²ACCI/E²MEND: the failure can be attributed to missing admission, failed fusion, retrieval loss, filtering, context packing, or generation. When Deployment constraints apply, LiteMem tests which parts of this path can be represented as repository-native files with Git provenance.

Lifecycle stages and interfaces. The framework also defines a stage sequence. Sources first produce dialogue, device, and visual observations; admission and ingestion normalize them; representation stores L0/L1/L2/SM records, fused nodes, and optional procedural hooks; retrieval and context assembly select bounded evidence; feedback updates or invalidates memory; governed evolution changes only accepted policies; and deployment tests which obligations survive under lighter substrates. Appendix B contains the compact lifecycle-stage map, while Section 8 reports evidence levels.

Five typed payload families make this stage sequence explicit across module boundaries. `FusedEvent` and `PerceptionFact` records move source evidence into `MemStack`; optional `ProcedureEntry` records move from `MemStack` to context assembly; `DiagnosticSignal` moves from `MemStack` to `E2MEND`; and `StrategyArtifact` moves from `E2MEND` back to `MemStack`. Table 15 and the extended interface-contract table in Appendix B specify which fields must survive a boundary for provenance, diagnosis, and rollback to remain possible.

Design rationale. The design principle is single-concern ownership. `MemFuse` handles device-stream causality, `MemSense` handles visual grounding, `MemStack` handles storage/retrieval correctness, `E2MEND` handles bounded strategy search, and `D2ACCI` handles evolution governance. Boundary crossings must pass through a typed payload. This separation enables two properties:

- **Independent evolution:** `E2MEND` can define a search space with up to 10^6 strategy combinations while limiting the risk of corrupting the retrieval framework (Section 6).
- **Incremental integration:** `MemFuse` and `MemSense` connect through typed payloads, allowing new source modalities to integrate with `MemStack` without changing the core storage and retrieval abstraction (Section 5).

Extended technical details—retrieval formulas, algorithm pseudocode, candidate traces, schema excerpts, error analyses, and protocol configuration—are collected in the appendix.

4 Structure: Structured Memory Substrate with `MemStack`

The **Continuity** requirement asks how later answers can reuse durable user state across turns without hiding which observations, memory items, and policy decisions justify that reuse. `MemStack` makes this requirement concrete as the runtime substrate of `Mi-Memory`: it turns histories and observations into typed records, preserves provenance across memory stages, and exposes retrieval, assembly, and invalidation traces for downstream governance. Later chapters build on this substrate by admitting richer evidence, evolving its policies, and testing whether the same audit obligations hold in lighter deployment settings. Optional procedural hooks share the same substrate, but they remain design-only and are not evaluated as standalone modules in this report.

At a glance

Role	RQ1 — Continuity / Structure
System role	<code>MemStack</code> runtime substrate
Contract	typed L0/L1/L2/SM records plus retrieval, assembly, and invalidation traces
Evidence	controlled-reference memory QA; optional procedural hooks remain design-only

4.1 Design Motivation: Continuity as an Audit Obligation

A personal AI assistant can lose continuity at several stages: it may fail to write the relevant event, retrieve a semantically adjacent but wrong memory, discard correct evidence during filtering, or answer incorrectly despite having the needed evidence. These failures motivate the central invariant of the **Structure** role:

Evidence should be written at the appropriate granularity, remain distinguishable across time, be recoverable through complementary search paths, survive filtering and invalidation, and fit within a bounded context budget.

This invariant is evaluated through traceable evidence flow, not a single universal threshold. Retrieval recall and context preservation are run-level validation checks interpreted within each benchmark slice; concrete values and formulas are reported as implementation details in Appendix B. The claim is architectural: continuity becomes diagnosable when memory stages emit inspectable artifacts.

4.2 Method: Trace-Producing Memory Runtime

The runtime is organized as an observable serving path, not a single opaque retrieval call. Its contribution is a joint algorithmic and systems formulation: memory organization, retrieval composition, and update are defined so downstream evaluation can attribute gains and failures to specific stages. Figure 3 illustrates the path: adapters normalize task inputs, layered storage maintains typed state, retrieval recovers candidate evidence, context assembly packs bounded evidence, and diagnostics record where evidence is retained, transformed, or dropped.

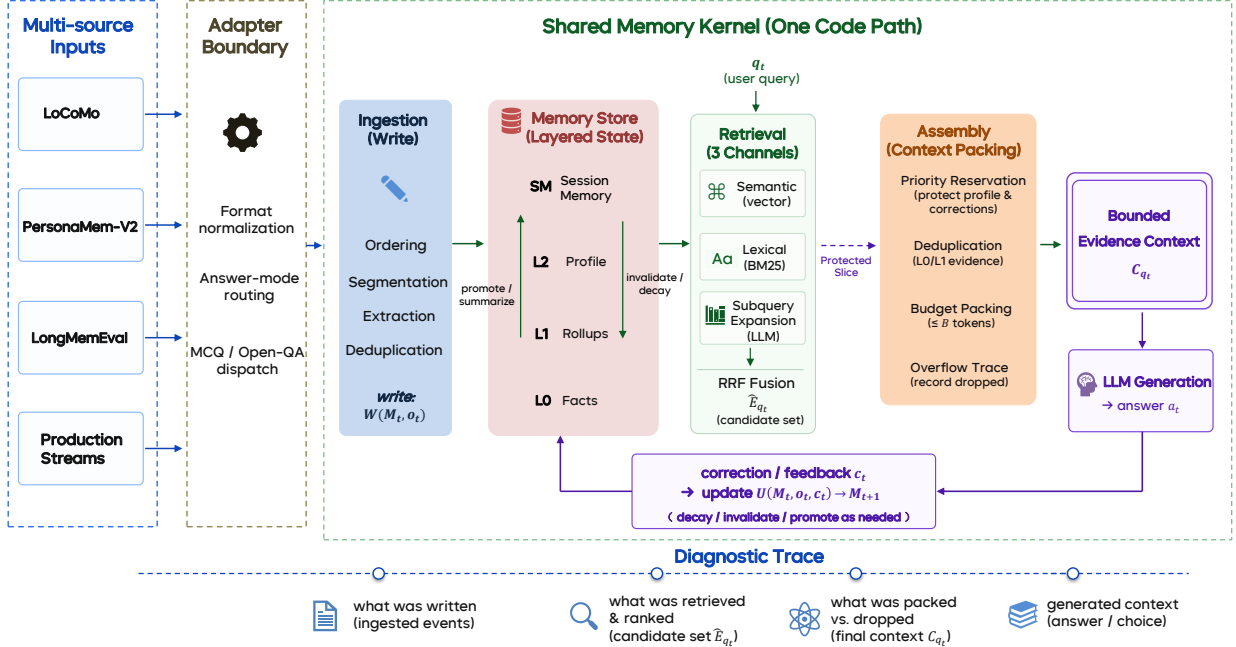


Figure 3 MemStack runtime.

The shared serving path has two purposes. First, benchmark-specific behavior is encoded as configuration on top of a shared kernel instead of separate code paths. Second, the same stages produce comparable traces across LoCoMo, PersonaMem-V2, and LongMemEval, so later D²ACCI/E²MEND analysis can attribute failures to stage-level causes rather than benchmark scripts.

Layered state. The runtime uses a four-layer operational hierarchy so evidence loss stays diagnosable, not opaque. **L0** atomic facts support precise grounding; **L1** session or topic rollups preserve coverage; **L2** profile entries support stable personalization; and **SM** session memory maintains unresolved local context. The layers are not proposed as a cognitive model. They are an observability decomposition, so a failure can be localized to missing event capture, over-compression, stale profile updates, or session-boundary handling. Detailed field formats, retention knobs, and retrieval parameters are deferred to Appendix B.

Ingestion, retrieval, and assembly. The serving contract has one rule: every admitted observation keeps temporal order, source identity, and a path back to the evidence that justified it. Long or topic-shifting histories may use supplemental extraction, but this chapter treats it as an implementation detail of the same admission contract; batching, segmentation, and concurrency details are reported in Appendix B. Retrieval combines semantic, lexical, and subquery-expanded channels because each channel recovers a different failure mode. Context assembly then reserves high-priority profile and correction constraints, deduplicates L0/L1 evidence, and records overflow traces rather than silently dropping constraints. Together these stages create the audit path needed to distinguish ingestion failure, retrieval failure, assembly failure, and generation error.

Table 4 Layer contract.

Layer	Write trigger	Primary role	Observable trace
L0	Per-event capture	Atomic facts and temporal grounding	Event/source ids and provenance pointers
L1	Session/topic boundary	Coverage-preserving local summary	Summary lineage and supersession links
L2	Stable profile update	Long-horizon preferences and constraints	Profile diffs and correction history
SM	Current turn / open loop	Short-lived context and unresolved state	Session-window trace and overflow record

The layer contract stays compact: the chapter includes only the detail needed to localize later failures, while the precise schema and retention values remain in the appendix.

Formalization: memory organization, retrieval, and update. Let the operational memory state at turn t be $M_t = (L_{0,t}, L_{1,t}, L_{2,t}, SM_t)$, where each component is a layered memory store matching Figure 3: $L_{0,t}$ holds atomic fact records with source links, $L_{1,t}$ holds session or topic rollups, $L_{2,t}$ holds stable profile and correction entries, and SM_t (Session Memory) holds unresolved short-lived context for the current turn.

This tuple is the structured realization of the abstract M_t in Section 3. q_t is the same user-request symbol and $o_t \in O_t$ an individual admitted observation. An observation enters through a layer-aware write operator W that may create, summarize, promote, or invalidate records while preserving provenance:

$$M_t^{\text{write}} = W(M_t, o_t). \quad (1)$$

For a query q_t , three retrieval operators R (for *Retrieval*) combine complementary channels—semantic (R_{sem} , vector similarity), lexical (R_{lex} , BM25), and expansion-based (R_{exp} , LLM subquery rewriting):

$$\hat{E}_{q_t} = R_{\text{sem}}(q_t, M_t) \cup R_{\text{lex}}(q_t, M_t) \cup R_{\text{exp}}(q_t, M_t). \quad (2)$$

Here $\hat{E}_{q_t}$ is the candidate evidence set before budgeting, corresponding to the overview’s E_{q_t} after channel union but before assembly. The selected evidence is then budgeted and deduplicated into the assembled context:

$$C_{q_t} = A(\hat{E}_{q_t}, M_t; B), \quad (3)$$

where the assembly step keeps a reserved slice for profile corrections and session-memory constraints before packing the rest under budget B . The symbol C_{q_t} is the same bounded generation context as in Section 3. When correction or feedback c_t arrives after generation, the update operator

$$M_{t+1} = U(M_t, o_t, c_t) \quad (4)$$

can decay, summarize, promote, or invalidate records. This formalization is operational, not cognitive: it describes audit-visible state transitions, not a psychological model.

Running trace: Ethan’s training handoff. Continuing the scenario from Section 1.2, the current equipment reminder is recorded in L0, the unresolved trip context stays in SM, the repeated practice routine is summarized into L1, and the shoe-storage correction updates L2. At answer time, context assembly protects the correction record from being crowded out by lower-priority evidence, so the response can distinguish a current reminder from a stable profile rule. The trace exercises the full continuity path from writing through retrieval, packing, and update.

Optional procedural hooks. When a user-specific procedure is enabled, MemStack can attach a compact trigger–procedure record to the same context-assembly path. The hook is admitted only after factual retrieval passes provenance and confidence gates, and it is injected as operational guidance, not factual evidence. In this report, these hooks remain design-only and are not benchmarked separately.

Kernel-adapter boundary. The shared kernel owns ingestion, storage, retrieval, assembly, and diagnostics; adapters normalize benchmark input shape and answer mode. This boundary keeps PersonaMem’s MCQ logic, LoCoMo’s temporal categories, and LongMemEval’s per-question haystacks outside the memory kernel, so continuity improvements are not hidden inside benchmark-specific forks. Because this report does not provide separate benchmark evidence for optional procedural hooks, their representation details and diagnostic examples remain in Appendix B.1.

4.3 Validation: Controlled Reference for Continuity

MemStack is evaluated as the controlled-reference runtime anchor for the lifecycle framework, not as a universal leaderboard claim. The evaluation asks whether the shared kernel can remain competitive with a reproduced reference while adding stage-local traces for diagnosis. External systems such as Mem0, AMA, MemMachine, DCPM, and HMO [7, 10, 13, 22, 34] remain relevant context, but their public numbers use different models, judges, prompts, and sampling policies.

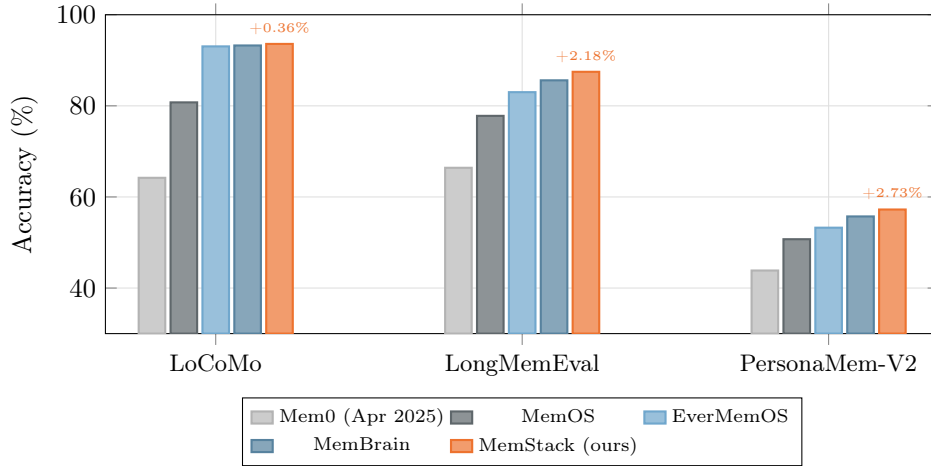


Figure 4 Controlled-reference comparison on three benchmarks (GPT-4.1-mini backbone). All baselines share the same evaluation harness from the EverMemOS evaluation; Mem0 is labeled by the Apr 2025 release of the original Mem0 paper. Orange labels report MemStack’s relative improvement over MemBrain on each benchmark.

Figure 4 provides the controlled-reference anchor: MemStack and the reproduced MemBrain baseline share the same harness, model roles, data splits, and diagnostic vocabulary. The associated diagnostic surfaces are: BM25+RRF and agentic expansion (LoCoMo), supplement path and forget constraints (PersonaMem-V2), and agentic retrieval with session cleanup (LongMemEval). The narrow LoCoMo margin reflects parity-level competitiveness, while LongMemEval and PersonaMem-V2 report larger relative gains over the controlled MemBrain reference. Representative failure-fix trajectories and case-level traces are preserved in Appendix E.

Main implication. MemStack establishes the continuity stage of Mi-Memory as both an algorithmic and systems substrate. User state can be represented, retrieved, assembled, invalidated, and audited through a shared memory kernel rather than an opaque RAG component. Its contribution is to expose multi-granularity state, hybrid retrieval, budgeted packing, and stage-local diagnostic traces within one controlled-reference setting. This role also explains the ordering of the later modules: Evidence Admission modules emit payloads into the same continuity substrate with provenance, D²ACCI/E²MEND modify policies through auditable strategy artifacts, and LiteMem is evaluated by how much of the same continuity contract survives in a repository-native substrate.

5 Expansion: Evidence Expansion with MemSense and MemFuse

The **Evidence Admission** requirement asks how observations beyond dialogue can become admissible memory evidence. For Personal AI, memory extends beyond a larger text corpus: evidence arrives through images, phones, cars, wearables, cameras, home devices, and tools as well as through dialogue. MemSense and

MemFuse form the Evidence Admission layer for RQ2. MemSense converts real-world scenes into structured, identity-preserving memory objects; MemFuse aligns atomic events from different devices, sessions, and times into cross-device causal evidence for MemStack. The shared requirement is provenance: additional sources are useful to the extent that their identity, time, device provenance, confidence, and relation to the user query remain inspectable after retrieval.

The admission path runs from real-world scenes and device observations $\rightarrow$ multimodal and event-level atomic memories $\rightarrow$ cross-device fused packs $\rightarrow$ structured assistant memory. As the source-side counterpart to MemStack, this layer expands what can count as admissible evidence before storage and retrieval: multimodal memory grounds visual context, while MemFuse connects evidence observed through different entry points.

At a glance

Role	RQ2 — Evidence Admission
System role	MemSense/MemFuse bridge
Contract	IKB entries and MemoryPacks preserve identity, time, provenance, confidence, and cross-event links
Evidence	MemSense evidence plus preliminary MemFuse evidence

5.1 Design Motivation: World Evidence Beyond Dialogue

Dialogue-only memory is insufficient in multi-device Personal AI ecosystems such as Human-Car-Home. User state is split across complementary sources: dialogue captures intent and self-reports, while perception and device events capture implicit state and environmental evidence. A restaurant destination selected on the phone may shape car navigation; a wearable sleep signal may contextualize a family routine; and an in-car reminder before a child’s training session may depend on a home-camera observation, a calendar event, route timing, and prior user corrections, rather than any single dialogue turn.

This split creates two evidence-admission gaps. The first is a **multimodal identity gap**: images and visual contexts need stable representation as evidence objects, not disposable attachments or unstable captions. The second is a **cross-device fusion gap**: related observations may describe the same situation without sharing keywords. If a home camera sees a training bag near the doorway, a calendar records basketball practice, and the vehicle estimates that returning home would still preserve on-time arrival, no single event is sufficient; the assistant needs to recover the temporal-causal chain before it can issue an in-car reminder.

Let O denote the typed observation universe used at serving time in Section 3: text, images, and device events are all observations, but each keeps its own payload schema and provenance fields. We write q_t for the current user request and e for an individual candidate evidence item. The Evidence Admission objective therefore selects an evidence set E_{q_t} by balancing two per-item terms: relevance to q_t and provenance strength for auditability.

$$E_{q_t} = \arg \max_{E \subseteq O \cup G \cup K} \sum_{e \in E} (\text{rel}(e, q_t) + \lambda \text{prov}(e)) \quad \text{s.t.} \quad \sum_{e \in E} \text{tok}(e) \leq B, \quad (5)$$

where E_{q_t} matches the selected-evidence notation in Section 3, G is the cross-device memory graph, K is the structured Image Knowledge Base (IKB), B is the answer-time evidence budget, λ balances relevance against provenance, $\text{rel}(e, q_t)$ scores how well a candidate evidence item supports the current request, and $\text{prov}(e)$ is the traceability term that rewards source identity, time/session metadata, device provenance, and causal links. Thus, the objective prefers evidence that is both on-topic and auditable, while still fitting within the budget. The exact scoring function is implementation-specific; the equation states the selection contract expected of this layer before downstream retrieval or generation. In Section 3 terms, this layer expands the admissible evidence pool inside O while keeping the selected subset E_{q_t} and the assembled context C_{q_t} governed by the same audit contract.

5.2 Method: Evidence Admission Pipeline from Grounding to Fusion

The pipeline is organized around one question: can a non-dialogue observation become assistant memory without losing traceability? MemSense grounds individual scenes and image observations as stable, identity-preserving evidence objects. MemFuse then decides whether related observations from different devices, sessions, or times should stay atomic or be fused into a MemoryPack with explicit provenance and causal edges. Both components feed MemStack through typed payloads instead of bypassing the memory runtime.

The stages are deliberately sequential. Evidence that cannot be named or traced should not be fused into higher-level memory; once evidence is fused, the resulting pack must still expose the atomic observations that justified it. This design separates source construction from answer-time selection: source modules expand the evidence pool, while MemStack and its diagnostic traces still decide which evidence reaches the final context.

5.2.1 Grounding Multimodal Evidence as Atomic Memory

For each image x_j , MemSense assigns a stable conversation identifier such as D3:IMG_001, a session s_j , a date $date_j$, a visual caption c_j , and a conversation-specific name n_j extracted from surrounding dialogue. The implemented MemSense pipeline (Figure 5) produces complementary stores inside MemStack. L0/L1/L2 and Session Memory anchor images to dialogue facts and temporal context. The IKB records image ID, path, date, session, conversation name, category, and related facts for deterministic counting, filtering, and name recovery. Multimodal retrieval indexes then provide probabilistic text–text, image–image, and text–image search.

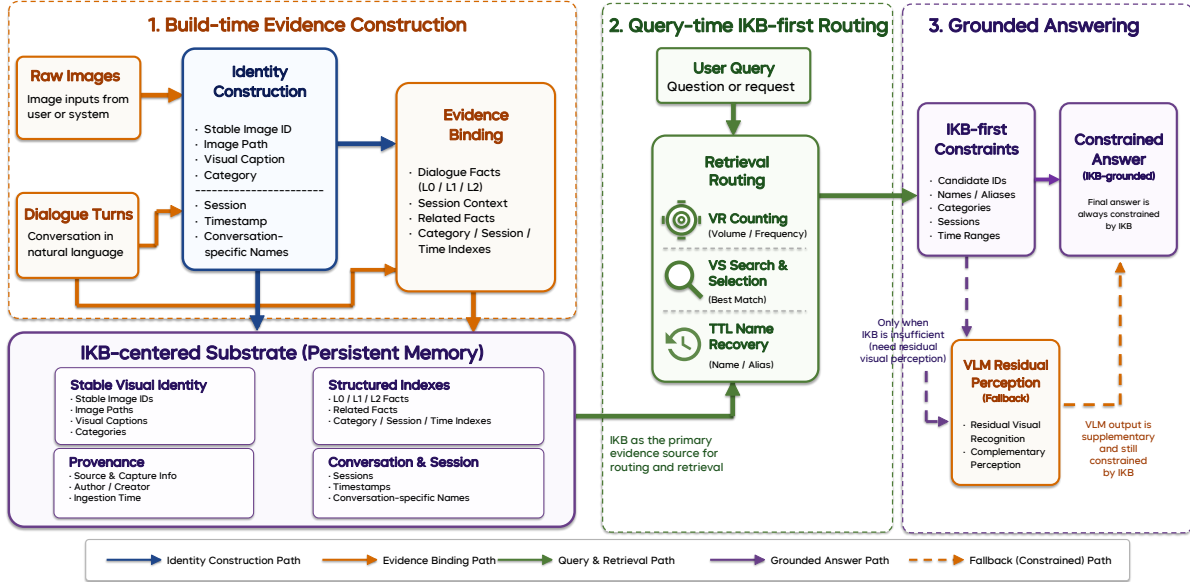


Figure 5 IKB path.

The IKB entry for an image is:

$$K(x_j) = (\text{id}_j, \text{path}_j, s_j, \text{date}_j, n_j, \text{cat}_j, c_j, \text{Rel}_j), \quad (6)$$

where cat_j is produced by a staged VLM/dialogue normalization pass, not by a fixed ontology alone. The builder extracts a visual category, reconciles conversation names and synonyms, and splits over-merged groups when image similarity is low. Rel_j is the set of related L0 facts selected by image-ID match, session/date overlap, and content-keyword match (written Rel rather than R to avoid overloading the retrieval operators $R_{\text{sem}}, R_{\text{lex}}, R_{\text{exp}}$ in Section 4). In the implemented prototype, IKB is a structured side table indexed by image id, session/date, category, and name, while L0/L1/L2 remain the general memory store. The two stores are linked by stable IDs instead of being collapsed into a single vector index. This binding matters because many

multimodal memory questions ask what was shown, counted, or named, not what a standalone VLM would label in isolation.

At answer time, the intent router maps visual-memory questions to execution strategies such as high-recall retrieval, date-filtered retrieval, session-direct retrieval, or `ikb_query`. The routing rule is:

When structured IKB records can resolve a visual identity question, they should constrain probabilistic visual recognition rather than only supplement it.

As a result, visual recall/counting (VR) uses IKB counts and image IDs, visual search/enumeration (VS) uses category/session/time-filtered candidate lists, and text-to-look naming (TTL) uses conversation-specific names before residual VLM perception.

5.2.2 Fusing Cross-Device Evidence into MemoryPacks

MemFuse extends the same evidence-preservation principle to asynchronous device events. Perception modules and dialogue devices emit observations as `AtomicEvent` records with provenance, time, importance, urgency, and visibility metadata. For each incoming event e_t , MemFuse retrieves candidate neighbors, expands them through graph links, and sends the neighborhood plus session context to an LLM judge. The judge chooses one of three actions:

$$\text{Fuse}(e_t, \mathcal{N}_t) \in \{\text{CreatePack}, \text{UpdatePack}(p_k), \text{Standalone}\}, \quad (7)$$

where $\mathcal{N}_t$ is the retrieved neighborhood and p_k denotes an existing `FusedNode`. `CreatePack` forms a new activity-level node from the incoming event e_t and its retrieved neighborhood; `UpdatePack` attaches e_t to an existing pack; and `Standalone` keeps e_t as an isolated atomic event when evidence for fusion is insufficient. This explicit three-way outcome keeps fusion conservative: ambiguous events remain recoverable as atomic evidence instead of being forced into a premature `MemoryPack`.

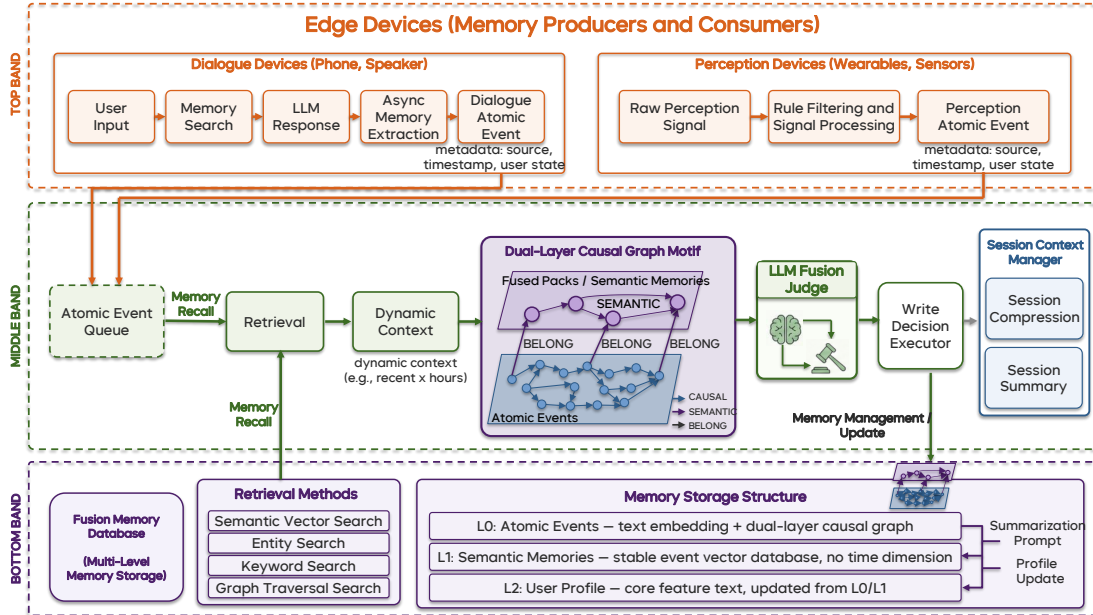


Figure 6 MemFuse pack path.

MemFuse uses session-level fusion, not a stateless pairwise classifier. It maintains a three-zone `FusionSession`: recent daily summaries, same-day accumulated events, and the current event with retrieved candidates. These zones are configuration-level budgets: when a zone is full, lower-confidence or older candidates are summarized or evicted, while accepted graph edges remain stored.

Accepted judgments are stored as a dual-layer graph (Figure 6). **AtomicEvent** nodes preserve fine-grained provenance; **FusedNode** nodes store activity-level packs, namely the **MemoryPacks** consumed by **MemStack**. **CAUSAL**, **SEMANTIC**, and **BELONG** edges capture causal links, retrieval neighbors, and event-to-pack membership. The processing cycle is: initialize zone state, collect candidates, judge fusion, persist the graph and **Memory-Pack**, then summarize or evict transient zone contents. Additional ingestion and session-graph diagrams are provided in Appendix C, so the chapter can stay focused on source-to-memory logic.

At query time, **MemFuse** searches atomic events and fused nodes with vector and BM25 retrieval, merges them with reciprocal-rank fusion, and treats the merged results as seed nodes. Seeds expand along **CAUSAL**, **SEMANTIC**, and **BELONG** edges, allowing “anything to check before training” to reach “training bag near doorway” despite limited lexical overlap.

In the running example (Section 1.2), graph expansion from the training-check intent reaches the fused pack that links bag, calendar, and route evidence through relation edges. Each fused node occupies one context slot but can expand into exact device and dialogue evidence when **MemStack** needs provenance. Table 5 summarizes how **MemSense** and **MemFuse** preserve the RQ2 invariants—identity, time, provenance, and causal relation—across the Evidence Admission layer.

Table 5 Evidence invariants.

Invariant	MemSense	MemFuse
Identity	Stable image ids, conversation names, IKB entries	AtomicEvent ids and FusedNode / MemoryPack ids
Time	Session/date fields linked to visual records	Event timestamps, daily/session zones, temporal graph edges
Provenance	Image path, dialogue context, related L0 facts	Device source, visibility metadata, source-event links
Causality / relation	Related-fact links for visual context	CAUSAL, SEMANTIC, and BELONG edges
Downstream audit	IKB-to-L0/L1/L2 links	MemoryPack payloads consumed by MemStack

5.3 Validation: Evidence Coverage in Realistic Daily Scenarios

The two components cover complementary sides of Evidence Admission. In this report, RQ2 covers construction and provenance preservation under module-specific harnesses: **MemSense** tests the grounding half of the admission contract, while **MemFuse** tests whether grounded observations remain connected across devices and time.

MemSense is evaluated on **Mem-Gallery** [1], a public benchmark for multimodal long-term conversational memory in MLLM agents. **Mem-Gallery** contains 240 multi-session dialogues across 20 scenarios, 3,962 dialogue rounds, 1,003 contextual input images, and 1,711 QA pairs with annotated clues. The result is treated as module-level evidence for the current IKB-centered multimodal memory pipeline.

Table 6 Mem-Gallery results (**MemSense**: GPT-4.1-mini backbone, GPT-4o-mini judge).

Metric / Error Source	Value	Scope	Interpretation
Average judge accuracy	89.15%	1,711 Qs	Primary metric; average fraction of correct votes across three judge runs
Strict binary accuracy	88.19%	1,711 Qs	Counts questions with judge accuracy below 1.0 as wrong
Strict errors	202	20 scenarios	Includes 34 partially correct borderline cases
Average latency	35.3s	Offline module pipeline	Median 32.4s; P99 91.8s
Harness processing failure rate	0%	Offline module pipeline	No offline harness processing failures observed
Image-related error share	58%	VR + VS + TTL	Dominant localized residual error source

The 89.15% and 88.19% figures (Table 6) differ only in aggregation: the former keeps the fractional three-vote judge score, while the latter applies a stricter binary rule and treats the 34 partially correct cases as errors. For reference, the **Mem-Gallery** paper [1] reports the best-performing open-source baseline (**MuRAG**) at 82.29% overall under a unified Qwen-2.5-VL-7B backbone, followed by **UniversalRAG** (80.16%) and **NGM**

(78.61%). Figure 7 shows these reference numbers. Direct numerical comparison is not claimed because the pipeline and backbone differ; the difference is treated as contextual support for the hypothesis that IKB-first structured routing may improve on end-to-end multimodal retrieval approaches. A controlled ablation isolating the IKB contribution under a shared backbone remains future work. The latency row reports offline pipeline latency under the evaluation harness, not production serving latency; environment details are not reported here.

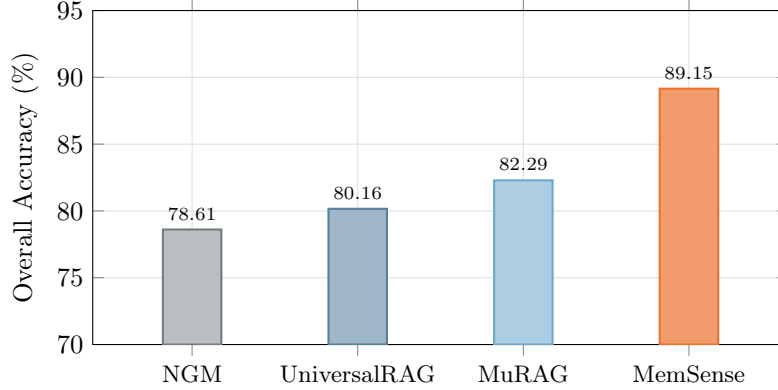


Figure 7 Mem-Gallery overall accuracy comparison. Baselines use a unified Qwen-2.5-VL-7B backbone as reported in [1]; MemSense uses GPT-4.1-mini with GPT-4o-mini judge. The backbone difference precludes direct numerical ranking; the figure provides contextual positioning only.

The residual errors are stage-localized: image-category construction can over-merge names, cross-session visual recall can truncate candidate sets, and answer-time VLMs can override conversation-specific labels with general visual names. The development trajectory and representative VR/VS/TTL case traces are moved to Appendix D. The operational conclusion is that IKB should be the primary evidence source for structured visual questions, with VLM perception reserved for residual disambiguation. The latency row also sets a deployment boundary: the current multimodal path provides module-level support for evidence construction, but production use still requires caching, trigger narrowing, and fast-path routing for common visual queries.

Validation is organized around the evidence-admission contract, not a general multimodal leaderboard. MemSense is judged by whether structured IKB records preserve identity, time, provenance, and downstream audit links; MemFuse is judged by whether cross-device packs recover causal chains and conflict relations without erasing contradictory evidence. MemFuseBench therefore checks whether admissible evidence survives the transition from raw observation to typed memory.

MemFuseBench follows a Scene-to-Sensor paradigm: it first defines what happened, then generates what each device observed. It is designed to be system-agnostic in execution: the same question set and checklist judge apply to Direct LLM, Naive RAG, mem0, and MemFuse, and scores reflect a common gpt-4o-mini judge pipeline.

Checklist scoring decomposes each answer into required evidence items: correct event identification, device-source attribution, causal link recovery, conflict handling, and final answer consistency. The judge marks each item before averaging into a dimension score; the rubric requires item-level yes/no decisions before the final score, not a free-form preference comparison. Because MemFuseBench is internal, these results are preliminary descriptive evidence for causal/fusion behavior, not an external-validity claim; external validation remains pending. The benchmark does not report a human upper bound, inter-annotator agreement, or oracle-fusion ceiling here, so absolute checklist scores indicate task-specific coverage instead of normalized task competence.

Within that boundary, the dimension breakdown in Table 7 suggests better checklist coverage for MemFuse k=20 on cross-device evidence construction. It reports the highest causal-reasoning, information-fusion, perspective-difference, and overall checklist scores among the compared systems, with a +4.7pp margin

Table 7 MemFuseBench scores.

System (4o-mini)	Causal Reasoning	Information Fusion	Conflict Arbitration	Multi-User Synthesis	Cross-User Query	Perspective Difference	Overall
Direct LLM	34.7%	31.6%	37.3%	32.3%	25.6%	21.2%	30.2%
Mem0 infer k=20	37.4%	31.8%	45.3%	26.9%	34.6%	15.0%	30.5%
RAG k=20	33.3%	24.4%	41.3%	31.4%	28.8%	16.3%	27.9%
MemFuse k=20	41.4%	42.2%	33.3%	25.5%	34.4%	30.6%	35.2%

relative to the mem0 reference. Scores in this range (30–42%) reflect granular sub-criterion coverage: each query is decomposed into multiple required evidence items, and partial credit is awarded per item.

The lower-scoring dimensions are also informative. MemFuse scores lower on conflict arbitration than mem0 (33.3% vs. 45.3%) and lower on multi-user synthesis than Direct LLM (25.5% vs. 32.3%), suggesting that the current graph-fusion objective over-prioritizes causal/link coverage and under-models contradictory evidence ownership, recency, and user attribution. Cross-user query is near parity with mem0 (34.4% vs. 34.6%), while perspective difference shows the largest observed MemFuse margin (30.6% vs. the next-best 21.2%).

Takeaway. MemSense and MemFuse expand the input surface of MemStack while keeping its core abstraction unchanged. Visual scenes become atomic memory objects with stable IDs and conversation names; device streams become atomic events with provenance and visibility; fused packs connect these observations across devices and time. The contribution is to keep expanded evidence typed, traceable, and auditable.

6 Evolution: Governed Evolution from D²ACCI to E²MEND

A memory system cannot remain fixed as user behavior drifts, new sources appear, model endpoints change, and previously effective heuristics become stale. Once memory is treated as lifecycle infrastructure, improvement cannot be reduced to an untracked prompt tweak or retrieval edit. A policy change should be versioned, evaluated against fixed evidence, and accepted through gates and rollback records. This section defines the governance plane of Mi-Memory: serving-time traces become evidence supporting or rejecting policy updates, D²ACCI supplies the human-governed diagnostic loop, and E²MEND automates only the bounded strategy-search subset of that loop. The design draws on trace-driven multi-agent evolution patterns such as AEGIS in HarnessX [5], but the claim here is narrower: evidence-gated strategy evolution for memory systems.

At a glance

Role	RQ3 — Evolution
System role	governance plane
Contract	Diagnostic traces, strategy artifacts, gate reports, and rollback records bind each policy change
Evidence	fixed-harness E ² MEND strategy search under the human-governed D ² ACCI loop

Control boundary. The E²MEND gates and rollback records address a specific strategy-drift risk. They reject schema-illegal, unversioned, or degrading strategy artifacts within the fixed offline harness before the serving strategy is reloaded. Adversarial model safety, privacy enforcement, consent management, and abuse resistance remain platform-level validation layers outside the current evidence scope.

This separation clarifies authority: D²ACCI decides what may change and how evidence is judged; E²MEND searches only the declared strategy space under those rules.

6.1 Design Motivation: Manual Diagnostic Iteration Cannot Keep Pace

The motivation starts with the bottleneck that makes bounded automation useful.

MemStack and D²ACCI establish the controlled memory runtime and diagnostic loop used in this report. The remaining bottleneck is scaling that diagnostic cycle. User behavior drift, new scenario onboarding, and

model upgrades all require engineers to re-run the diagnostic-fix-verify cycle. As scenario coverage grows, the manual loop becomes more costly to sustain.

E²MEND automates the repeatable portion of D²ACCI: diagnosis, strategy proposal, and gated verification under a locked framework and schema-constrained strategy space. It does not modify the framework, evaluation protocol, data schema, or governance criteria defined by D²ACCI. The trade-off is deliberate: E²MEND can tune parameters, prompts, routing rules, and feature toggles, but any change that expands the capability boundary returns to human-governed D²ACCI review.

E²MEND therefore differs from recent deep-research agents that learn from trajectory or case memory on-line [27, 45]: evolution is treated as a governed strategy-artifact update, not unconstrained serving-time policy learning.

The next subsection defines the dual-loop boundary that separates human-governed capability changes from automated strategy search.

6.2 Method: Dual-Loop Framework for Bounded Evolution

Bounded evolution starts by separating mutable strategy from locked framework. The two loops divide responsibility: D²ACCI is the governance loop for framework, protocol, data schema, and acceptance criteria, while E²MEND is the automation loop for mutating declared strategy artifacts. This distinction supports regression control and reproducibility: when a candidate is accepted, the system can attribute the gain to a declarative strategy mutation rather than hidden framework drift.

Table 8 Dual-loop division of labor.

Scope	D ² ACCI governance loop	E ² MEND automation loop
Declarative strategy space	Defines admissible fields, value ranges, invariants, and evolvable dimensions	Mutates extraction limits, retrieval weights, top_k, rerank size, source windows, prompts, and feature toggles
Framework	Approves locked components: extraction code, retrieval pipeline, storage logic, context assembler	Reuses the locked framework without changing code or storage schema
Protocol	Defines benchmark data, judge policy, scoring rubric, gate criteria, and non-regression rules	Runs the fixed slice, applies gate checks, and reports paired deltas under those rules
Artifacts	Requires versioned artifacts, traces, gate reports, and rollback records for audit	Persists each candidate mutation, decision record, revert, and restore event

This boundary is motivated by an internal qualitative audit of 13 baseline memory providers. The reviewed systems typically package extraction, retrieval, pruning, and presentation into 200–500 lines of tightly coupled provider logic. As a result, changing one strategy often means rewriting the provider, which complicates A/B testing, rollback, and auditing. The capability matrix suggests that simultaneous support for serving-time memory updates, explicit failure learning, memory-attribution traces, and consolidation, not append-only growth, is still uncommon among the reviewed providers. The dual-loop design reframes this point as a division of labor: D²ACCI governs capability and evidence contracts, while E²MEND executes bounded strategy automation against those contracts.

The division of labor in Table 8 also defines the handoff rule: if a candidate requires framework code, benchmark protocol, storage schema, or acceptance criteria to change, the issue is handed from E²MEND automation to D²ACCI governance. Autonomous acceptance is limited to candidates that remain within the declared strategy space and pass bounded regression checks.

Figure 8 makes the coordination explicit. D²ACCI and E²MEND are two loops with distinct responsibilities and implementation paths. D²ACCI decides what kinds of changes are legitimate and how evidence is judged. E²MEND repeatedly tests strategy mutations under those fixed rules, preserving the artifacts needed for audit and rollback.

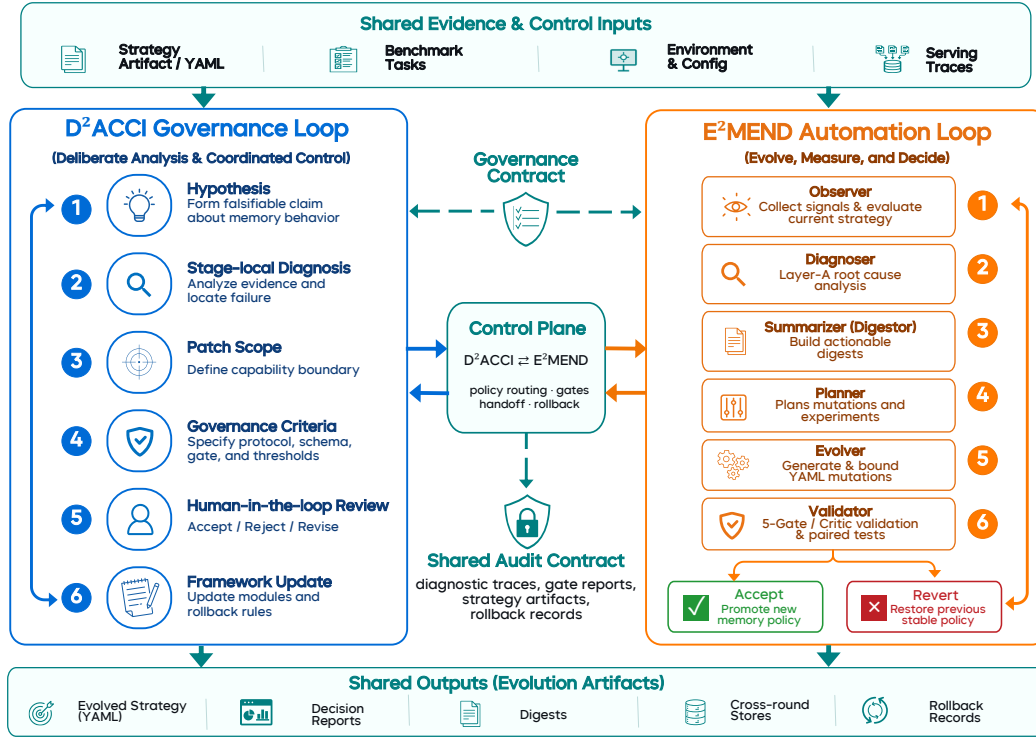


Figure 8 Dual-loop framework for governed memory evolution.

6.2.1 D²ACCI: Human-in-the-Loop Evolution Governance

Within this boundary, D²ACCI (Diagnostic-Driven AI-Collaborative Iteration) is the human-in-the-loop governance loop for memory evolution. Humans define falsifiable hypotheses, AI agents help scale diagnosis and patch synthesis, and deterministic analyzers decide whether the resulting evidence justifies acceptance. This subsection formalizes the evidence contract that E²MEND later automates within a narrower strategy space.

The method rests on two invariants. First, *stage-local attribution*: when a memory-dependent answer fails and sufficient evidence annotations are available, the diagnostic trace localizes the earliest evidence-loss stage among ingestion, retrieval, filtering, context assembly, and generation (formalized in Appendix E). This avoids misdirecting retrieval fixes toward ingestion problems or generation fixes toward filtering problems. Second, *constrained improvement*: a candidate configuration is accepted only when paired comparisons show positive evidence under the applicable gate and every question category satisfies its non-regression threshold. If a run lacks an explicitly reported p-value, confidence interval, or variance estimate, the comparison is treated as descriptive evidence only.

Iteration loop. A D²ACCI round proceeds in four steps:

1. **Hypothesis:** An engineer proposes a falsifiable claim (e.g., “single-hop temporal questions fail because BM25 cannot resolve date expressions”) and predicts which failure category should decrease.
2. **Diagnosis:** AI agents run the current harness, produce per-question diagnostic traces, and the Layer-A classifier routes each failure to the earliest responsible stage—ingestion, retrieval, filtering, or generation—when evidence annotations and source-id links are available.
3. **Patch:** The engineer (or an AI agent) implements a bounded fix scoped to the hypothesized stage (e.g., adding date-aware BM25 tokenization), keeping the change within one module boundary.
4. **Verification:** The analyzer aligns baseline and candidate outputs by question key, partitions samples into improved / regressed / both-wrong / both-correct, and applies the configured gate. If the hypothesis

prediction matches the observed category movement and no category regresses beyond threshold, the patch is accepted.

Concrete example. In one representative round on the LoCoMo development trace, the Layer-A classifier attributed 30% of remaining failures to `ingestion_gap`—cases where relevant dialogue turns were never extracted into L0 memory. The hypothesis was that long conversations (>50 turns) exceeded the single-pass extraction context budget, silently dropping later evidence. The fix—adding the supplement extraction path with topic segmentation and overlapping windows—reduced `ingestion_gap` from 30% to 2% of residual failures and yielded a cumulative +4.48pp accuracy gain. The paired gate reported per-category non-regression, and rejected alternative directions (aggressive prompt expansion, threshold lowering) were archived as negative priors for future E²MEND search.

Verification infrastructure. Given baseline and candidate outputs aligned by question key, the analyzer partitions samples into improved, regressed, both-wrong, and both-correct sets. A candidate first passes a bounded probe; when the run reports sufficient paired evidence, the full gate can include McNemar-style checks, bootstrap intervals, and per-category non-regression gates. When those statistical artifacts are not reported, the result is used only as descriptive development evidence. Fixes should be implemented in shared modules whenever possible, enabling a unified framework without benchmark-specific forks. Extended formulas, trace schemas, and representative iteration rounds are provided in Appendix E.

Empirical validation. The representative development trace (Table 24 in Appendix E) indicates controlled improvement: most accepted gains came from fixing a localized failure surface, not adding broad prompt complexity; rejected directions were archived as negative priors for later automated search.

Limitations. D²ACCI is most reliable when failures are observable at pipeline stages and intermediate artifacts are recorded. Its claims are scoped to stage-local diagnosis and human-approved hypotheses; it does not claim convergence or optimality. These limits motivate E²MEND: bounded, reversible strategy search under the same evidence and acceptance constraints, while framework or protocol changes remain in the human-governed D²ACCI loop.

6.2.2 E²MEND: Automated Strategy Evolution

E²MEND implements the automation side of the dual-loop framework. It does not treat the memory system as an unconstrained program-synthesis target. Its admissible search space is a *schema-constrained declarative strategy space* with three parts: a schema-level contract for admissible fields, value ranges, evolvable dimensions, and cross-field invariants; strategy artifacts that instantiate concrete policies for ingestion, retrieval, lifecycle management, presentation, and feature toggles; and symptom-triggered candidate bundles that map observed failure modes to interpretable policy moves. Prompts are evolvable only when represented as versioned, rollback-capable strategy artifacts. Implicit prompt edits embedded in framework code are treated as framework changes governed by D²ACCI, not strategy mutations handled by E²MEND.

Base harness and strategy space. The locked base harness contains components retained from repeated D²ACCI iterations: cross-encoder rerank, intent parsing/query decomposition, layered retrieval, and raw-dialogue backfill. These components remain in the framework core, reducing uncontrolled regression risk while preserving a fixed development baseline. Within that boundary, E²MEND can evolve only declared strategy fields:

- Extraction: versioned prompt templates, max facts per trajectory, dedup threshold.
- Retrieval: `top_k`, time-decay `half_life`, BM25/vector weight ratio, rerank `top_n`, intent overrides.
- Lifecycle and presentation: pruning score, compression policy, `source_window` size, output format.
- Advanced features: failure correction, conflict handling, conditional memory triggers.

Observe–Improve–Verify Loop. Each E²MEND evolution round uses a strategy artifact, benchmark tasks, and the environment configuration as inputs. A **Bootstrap Runtime** normalizes tasks, builds the evaluator, and initializes the EvolutionEngine. The loop then proceeds in three phases: Observe evaluates the current strategy and assigns stage-local root causes; Improve mutates only declared strategy dimensions; and Verify applies scope checks, Gate/Critic validation, targeted screens, paired comparisons, and rollback safeguards before accepting any serving update. The full per-round pseudocode and Layer A decision tree are kept in Appendix F.

The three-phase design keeps deterministic checks and gate validation on the critical path while confining exploratory mutations to the bounded YAML strategy space. In the implementation, rollback is triggered when current accuracy falls more than 3pp below the all-time best accepted strategy, separating candidate rejection from serving-strategy drift. Layer A routes failures into ingestion, retrieval, or generation gaps only when complete evidence annotations and source-id links are available; in deployment-like settings, the same logic is heuristic and audit-assisted, not a deterministic earliest-loss claim. The Critic adjusts strictness by actionability, and an optional UCB1 hypothesis tree can prioritize future mutation directions. These implementation-level rules are detailed in Appendix F.

Safeguards and rollback. E²MEND searches a textual strategy space, not a continuous parameter space. It faces risks analogous to RL policy optimization—for example optimizing a proxy metric, over-exploiting one dimension, or accumulating drift—but this report does not claim to prove RL-style reward hacking, policy collapse, or catastrophic forgetting in this setting. RL governance is therefore used only as an analogy for a three-layer defense (Table 9).

Table 9 Evolution safeguards.

Layer	Mechanism	Analogy	Risk Addressed
Hard constraint	5-Gate fail-fast pipeline	Action mask	Schema loopholes
Soft review	Reputation-driven Critic	Independent value estimate	Overuse of one mutable dimension
Cross-round safeguard	Best-ever rollback	Checkpoint restore	Accepted-strategy drift

Every candidate passes through a fail-fast gate pipeline followed by an independent Critic. The gates enforce schema legality, novelty, value ranges, prompt integrity, and replay stability; the Critic adds reputation-driven soft review before expensive evaluation. Rollback then operates at two levels:

- **Per-round revert:** If the best candidate’s delta falls below threshold, the strategy is not updated. The failure is logged to `dimension_effects`, increasing revert count and degrading that dimension’s reputation for future Critic checks.
- **Best-ever auto-restore:** Every round persists the all-time best strategy and accuracy. If current accuracy degrades by > 3pp below best-ever, the system restores from `best_strategy.json`—regardless of whether the current round accepted or rejected a candidate. This bounds accumulated drift across many rounds.

Detailed gate definitions and Critic heuristics are provided in Appendix F. Eight cross-round persistent stores carry journals, dimension effects, progress scoreboards, question histories, candidate outcomes, stage audits, Layer-A logs, and hypothesis-tree state. Together, they provide cross-round state for the Replay gate, Critic reputation, optional UCB1 direction search, and rollback safeguard.

6.3 Validation: Strategy Improvement under Bounded Automation

The evaluated subset covers the implemented parts of the dual-loop design: Framework/Strategy separation, Layer A diagnosis, Digester–Planner–Evolver proposal, Gate/Critic validation, targeted screening, accept/revert control flow, best-ever rollback, and cross-round persistent stores. Remaining roadmap items are deferred to Section 9. The claim boundary is limited to *bounded offline strategy automation* inside a locked harness; arbitrary memory-system evolution and online deployment are not evaluated here.

Experimental configuration. The run keeps the same role-separated offline configuration across stages: serving-side extraction/routing, evolution agents, and QA judging remain fixed, with full assignments listed

in Appendix F. The MemStack LoCoMo number in Section 4.3 is the earlier controlled-reference anchor for the base memory system. The E²MEND number reported here is a subsequent staged result under the same offline harness; it is evidence of governed strategy improvement over time, not a direct leaderboard comparison between independently timed runs.

On LoCoMo, E²MEND improves the initial memory-system baseline from **75.58%** (1164/1540) to the reported best checkpoint of **94.74%** (1459/1540), a +19.16pp gain and +295 net correct answers. This is descriptive full-benchmark evidence for the staged strategy loop, not a repeated-run statistical claim.

The improvement was staged rather than the result of a single prompt rewrite. Table 10 groups the accepted trajectory into external-facing strategy families; representative accepted, rejected, and no-effect candidate outcomes are preserved in Appendix F. Reproducibility is artifact-level: each E²MEND result is tied to a fixed benchmark, baseline, strategy artifact mutation, candidate trace, and gate record; exact replay may depend on model and judge endpoints.

Table 10 Compressed E²MEND strategy checkpoints on LoCoMo.

Strategy family	External-facing description	Stage	Acc.	Gain
Initial memory strategy	Baseline before staged E ² MEND evolution	S0	75.58	–
Evidence-grounded context enrichment	Attach local dialogue evidence and recover inventory-style facts	S1	81.82	+6.23
Structured state abstraction	Convert evidence into typed temporal, location, plan, preference, slot, and ledger state	S2	87.08	+11.49
Verified candidate adoption	Adopt only candidates that pass support, shape, and non-regression checks	S3	91.49	+15.91
Deterministic support-cluster decision policy	Cluster equivalent verified candidates and choose the best-supported answer	S4	93.70	+17.86
Evidence-supported candidate consolidation	Consolidate supported candidate answers while excluding low-confidence candidate classes	S5	94.74	+19.16

The largest absolute gains occur in single-hop (+139 correct), temporal (+69), and multi-hop (+70) questions. Inference remains the most difficult bucket, but it still improves by +17 correct. Beyond the score movement, the validation question is whether strategy updates are accepted, rejected, and archived under stable rules. The appendix trace indicates that E²MEND did not monotonically accept every proposal: self-correction and broad raw-dialogue fallback were rejected as regressive or unstable, while evidence admission, typed-state abstraction, verified candidate adoption, deterministic support clustering, and evidence-supported consolidation formed the accepted path.

E²MEND studies *how* memory strategies can be changed under evidence constraints. Together with D²ACCI, it instantiates the diagnostic-evolution category in Figure 1: the evidence-governed D²ACCI loop defines human-reviewed capability changes, while the bounded E²MEND loop searches the declarative strategy space under those fixed rules. This division of labor aligns with the independently proposed SSGM framework [16], which argues that memory evolution and governance should be decoupled to reduce drift and poisoning risk in deployed agents. In this section, “decoupled” means that the two loops have different responsibilities and implementation paths while sharing one audit contract. The resulting audit contract is explicit: a memory-system change must be represented as an inspectable artifact, evaluated under fixed governance, and then accepted, rejected, or rolled back with preserved evidence.

Takeaway. D²ACCI keeps diagnosis human-auditable, while E²MEND automates only bounded strategy search under a locked framework. The objective is to make memory evolution reproducible, comparable, and reversible under one governance contract.

Future: Online Incremental Evolution. This capability is not evaluated here. Production environments lack ground-truth labels, so online evolution should add narrowly scoped auxiliary prompts or conditional rules, trigger them only on matching data, track isolated impact, and merge or disable them independently.

7 Deployment: Lightweight Memory with LiteMem

The **Deployment Transfer** requirement asks whether the same audit contract can survive when memory moves from service-specific stores into local repository-backed state. LiteMem is used here as a substrate-transfer probe, not a complete replacement for the service-side stack. The substrate uses Markdown/YAML records, file-tool retrieval, and Git provenance, so continuity, editability, rollback, and bounded retrieval remain inspectable under local-first constraints. This positioning connects to repository-native and local-first memory tools, with related motivation from direct corpus interaction, Git/version-controlled context, and lightweight memory-augmented generation [2, 17, 25, 29, 37].

At a glance

Role	RQ4 — Deployment Transfer
System role	LiteMem repository-native substrate test
Contract	Markdown/YAML state, file-tool retrieval, Git provenance, and file-level audit traces
Evidence	LoCoMo-aligned transfer-feasibility evidence

7.1 Design Motivation: Memory under Deployment Constraints

Personal AI should not assume one cloud-hosted vector-store runtime. Cost, latency, privacy, offline operation, and user editability all push memory toward local or edge substrates. LiteMem asks a narrower transfer question: can a repository-native implementation retain a substantial fraction of the measured memory benefit while preserving the lifecycle framework’s audit obligations? The retained improvement is reported against the same no-memory baseline, with formulas and local decay settings in Appendix G. This is transfer evidence within a LoCoMo-aligned setting, not a production-scale edge deployment claim.

7.2 Method: Repository-Native Audit Substrate

LiteMem models memory as repository state, not a specialized service. Its design target is to keep recall, capture, organization, edit history, and rollback visible through standard file and version-control operations. Markdown/YAML files provide persistent state, file search provides the initial retrieval path, and Git history provides the provenance and rollback layer. If Git is unavailable, the system still exposes inspectable files, but provenance and rollback support is more limited.

Each record exposes three views: a machine-readable lifecycle state, a model-readable retrieval summary, and provenance links for audit. Figure 9 illustrates how these views participate in a repository-native lifecycle: query, recall, lazy loading, answer or action, capture, organization, and Git-backed history. The concrete directory hierarchy and YAML fields are implementation details reported in Appendix G. The claim tested here is that L0/L1/L2/SM-style information can map to local profile, session, entity, knowledge, and raw daily-event files without being hidden inside a database.

Table 11 LiteMem audit surfaces.

Surface	Representation	Main role
Summary files	title, summary, keywords, and frontmatter metadata	first-pass retrieval and ranking
Raw daily logs	append-only Markdown event records	repair path for missed writes
Profile/entity/knowledge files	consolidated long-term state and procedures	stable user state, organization output, and task routines
Git history	diffs, commits, and rollback traces	provenance, supersession, and debuggability

Repository surfaces in one view. The lightweight substrate is structured, not flat. In the evaluated design, one memory often maps to one Markdown file or one section inside a consolidated file; the repository then organizes four broader audit surfaces: compact summary files for first-pass retrieval, append-only daily logs for repairable capture, consolidated profile/entity/knowledge files for stable state, and Git history for

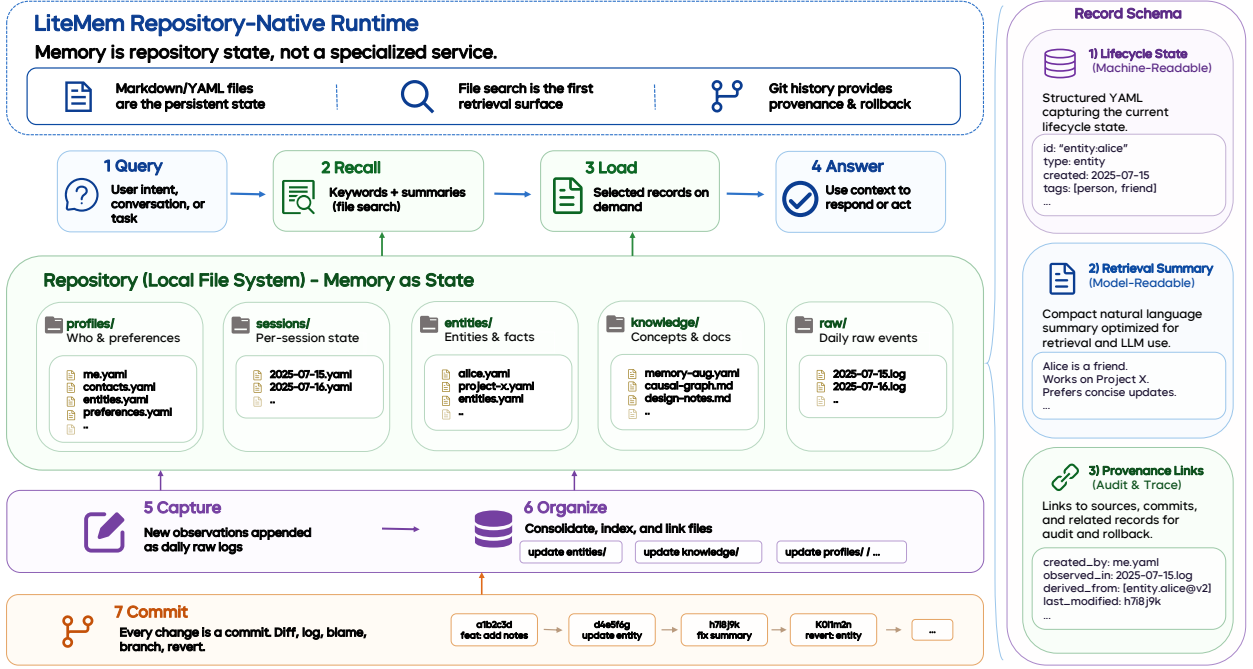


Figure 9 LiteMem recall path.

provenance and rollback. This separation makes the deployment-transfer claim inspectable: the system can search, inspect, edit, and audit memory with standard developer tools, not hidden service metadata.

Appendix G provides the schema details. Conceptually, the repository carries lifecycle responsibilities that a service-side memory stack would normally distribute across a database, index, and audit service.

Retrieval scoring and temporal decay. Candidates are ranked by lexical relevance, current importance, recency (Δt since last access, timescale τ), and a short-term access-feedback signal. The scoring function uses a memory record m , the current query q_t , and the current turn t as explicit arguments:

$$\text{Score}(m, q_t, t) = \lambda_1 \text{Lex}(m, q_t) + \lambda_2 \text{Imp}_t(m) + \lambda_3 \exp(-\Delta t / \tau) + \lambda_4 \text{Access}(m). \quad (8)$$

Here $\text{Lex}(m, q_t)$ is the lexical-match score, $\text{Imp}_t(m)$ is the current importance state, and $\text{Access}(m)$ is a normalized access-feedback feature. Idle organization recomputes the importance state (Eq. 9) and modulates it with user feedback (η , ρ are boost/penalty coefficients):

$$\text{Imp}_t(m) = \text{Imp}_0(m) \cdot \exp(-\Delta t_m / \tau_m) + \eta \cdot \text{access_count}(m) - \rho \cdot \text{skip_count}(m). \quad (9)$$

The two decay terms use different time bases: in Eq. 8, Δt is the interval since the record was last accessed (retrieval recency); in Eq. 9, Δt_m is the interval since the record’s importance was last assigned or refreshed (organization-time salience decay). The corresponding timescales τ and τ_m reflect this difference: τ is short (query-time ranking) while τ_m is longer (background organization).

Progressive disclosure and lifecycle. LiteMem uses *progressive disclosure*: retrieve compact summaries ranked by Eq. 8, select top- k candidates, and lazy-load full text only for selected records. The lifecycle has three flows: recall before answering, raw capture after interaction, and idle or session-boundary organization. The append-only daily log preserves a repair path when structured consolidation is incomplete. Details are in Appendix G.

Running trace: Ethan’s training handoff. In the running scenario (Section 1.2), LiteMem scans compact summary files for “training”, “bag”, and “equipment”, then lazily expands the matching daily log and entity

file. A recent correction—spare shoes kept at school—is visible in Git history, which records when it entered the repository, while the diff identifies which earlier statement it superseded. After the interaction, the new outcome is appended to the daily log and later consolidated into the entity file, exercising all four deployment surfaces: summary recall, raw-event repair, consolidated state, and repository provenance.

7.3 Validation: Substrate Transfer

For validation, the repository-native design is instantiated in a local agent runtime and evaluated with a modular TypeScript harness. The harness uses matched prompts across full-history, keyword retrieval, deterministic extraction, LLM extraction, and agent-flow variants. Implementation details such as atomic writes, temporal simulation, speaker attribution, and organization actions are summarized in Appendix G.

By separating retrieval, extraction, and flow-level choices under the same substrate, the harness keeps the result interpretable as audit-substrate transfer rather than a prompt-template or serialization artifact.

Experimental results. Table 12 reports transfer-feasibility evidence under a LoCoMo-aligned file-tool setting. LiteMem reaches 90.81%, retaining 90.0% of the service-side improvement over the same no-memory baseline, with a 97.0% absolute score ratio against the full MemStack reference. The 90.0% figure is computed relative to the same no-memory baseline: $(90.81 - 65.83)/(93.59 - 65.83) = 90.0\%$. The claim is substrate-transfer evidence within the aligned setting, not a leaderboard submission or production cost/latency curve.

Table 12 LiteMem transfer results: multi-dimensional comparison.

System	Accuracy	Infra. cost	Substrate	Latency	Trade-off summary
MemStack (full)	93.59%	High	Cloud service	Moderate	Vector DB + embedding + reranker; full diagnostic surface
LiteMem	90.81%	None	Local Markdown/Git	Low	90.0% retention; no external dependency; fully offline
No-memory baseline	65.83%	None	Full-history prompt	High	Long-context only; no memory organization
Raw chunk-retrieval	55.28%	None	Naive file-split	Low	No semantic structure; lowest recall in this comparison

The category profile clarifies where transfer holds: direct recall and temporal localization remain relatively strong, while multi-hop relation traversal is the main remaining limitation. The result supports transfer of audit discipline—representation, retrieval discipline, provenance, and organization policy—not equivalence to a full service-side memory stack.

Takeaway. LiteMem exposes two deployment boundaries. First, repository-native recall is limited by lexical search and lightweight organization unless optional local indexes or compact graph files are added. Second, long-horizon file-count scaling and latency remain unvalidated beyond the current transfer-feasibility setting. Within these boundaries, LiteMem makes the deployment side of the audit contract concrete: in the evaluated repository-native setting, memory stays inspectable, editable, and provenance-aware even when specialized memory infrastructure is removed.

8 Evidence-Governed Claims Across the Lifecycle

This section is a claim ledger rather than a benchmark summary. It asks whether the same evidence-governed audit contract remains coherent across Structure, Expansion, Evolution, and Deployment. Each role contributes one evidence anchor with a stated maturity level and claim boundary.

Evidence-governed audit contract

A memory result is admissible when it states what evidence entered the system, how that evidence was represented, which retrieval or governance decision used it, which trace makes the decision inspectable, and which claim boundary limits over-generalization.

Why these modules belong together. MemStack tests durable state; MemSense/MemFuse test multimodal and cross-device evidence admission; D²ACCI/E²MEND test governed memory-policy change; and LiteMem tests substrate transfer. The modules are heterogeneous by design, but they expose the same artifact families: typed evidence, provenance, diagnostic traces, gated updates, and deployment boundaries.

8.1 Evidence-Governed Evaluation

Because Mi-Memory studies a lifecycle framework, the shared protocol evaluates audit obligations, not a universal metric. Each track states its run setting, admissible evidence and provenance, available traces, governed-change criterion, and explicit claim boundary. Missing artifacts or preliminary tracks remain marked at lower evidence levels.

Evidence-level taxonomy. The report uses *evidence level* for evaluation maturity and *statistical qualifier* for repeated-run, confidence-interval, or paired-test support. Unless such support is explicitly reported, numerical differences are descriptive rather than significance claims. The levels used here are: *controlled reference* (MemStack vs. reproduced MemBrain), *module-level public or aligned* (MemSense), *controlled offline benchmark* (E²MEND), *transfer-feasibility* (LiteMem), *preliminary/internal* (MemFuse), and *design-only* (optional procedural hooks). Run configurations and resource accounting are collected in Appendix H.

Consolidated evidence across lifecycle roles. Each lifecycle role contributes one bounded evidence anchor. For **Structure**, MemStack reaches 93.59% / 57.24% / 87.47% on LoCoMo / PersonaMem-V2 / LongMemEval under a controlled-reference comparison with a reproduced MemBrain baseline; the +0.34pp LoCoMo margin is descriptive parity. For **Expansion**, MemSense reports 89.15% Mem-Gallery accuracy, while MemFuse reaches 35.2% on the internal MemFuseBench (+4.7pp over mem0@20), framed as preliminary cross-device fusion evidence. For **Evolution**, D²ACCI/E²MEND improve a subsequent locked LoCoMo offline run from 75.58% to 94.74% through traceable, gate-bounded strategy changes; this is not compared directly with the earlier MemStack anchor. For **Deployment**, LiteMem demonstrates 90.81% transfer score and 90.0% retention, establishing transfer feasibility rather than a scaling claim.

The results are role-specific, not a cross-role ranking. Their shared invariant is the audit contract: typed evidence, provenance, and an explicit boundary against over-generalization.

8.2 Cross-Module Findings and Limitations

The lifecycle view surfaces interactions that would be hidden in a module-only report:

- **E²MEND + MemStack:** E²MEND can tune retrieval strategy inside a locked MemStack framework because the runtime exposes strategy artifacts and diagnostic traces instead of hard-coded patches.
- **MemFuse + continuity substrate:** MemFuse outputs typed cross-device payloads that can be admitted into the same downstream memory contract used by MemStack and optional procedural hooks.
- **Source-filtering tradeoff:** source-side filtering can improve dialogue QA while risking the loss of low-similarity causal edges, so causal payloads require separate retention and evaluation criteria.

These compatibility observations are not a substitute for joint ablation.

The current evidence supports contract compatibility and calibrated per-module progress; marginal contribution still requires a joint ablation benchmark.

Limitations and pending experiments.

Safety and privacy boundary

The audit contract makes memory behavior inspectable; privacy enforcement remains a platform-level layer. Deletion propagation, cross-user leakage, memory poisoning, Git-history exposure, and visual sensitive-information handling require platform-level access control, consent management, and adversarial validation beyond the module-level evidence reported here.

The benchmarks exercise recall fidelity, visual grounding, cross-device causality, bounded strategy evolution, and repository-native transfer in separate tracks. They do not yet provide a closed-loop personal AI benchmark where memory guides downstream actions [12, 19], nor do they test extreme-scale context volumes [32]. Full end-to-end ablation, year-scale deployment, and platform-level privacy validation remain roadmap items.

9 Conclusion and Outlook

This report presents Mi-Memory as a lifecycle memory framework for personal AI. The central conclusion is that assistant memory should be treated as inspectable infrastructure rather than a retrieval layer alone: durable state, multimodal evidence, governance, and deployment constraints must remain jointly auditable.

Mi-Memory operationalizes this view through four roles. MemStack anchors Structure, MemSense/MemFuse support Expansion, D²ACCI/E²MEND govern Evolution, and LiteMem tests Deployment under a repository-native substrate. Their claims are tied to traces and paired comparisons when available, and otherwise bounded by the evidence-level taxonomy in Section 8.

Evidence-Bounded Takeaways

- **Memory as infrastructure:** Assistant memory is treated as a governed substrate for continuity, personalization, and cross-device grounding rather than a retrieval layer alone.
- **Audit contract as integration primitive:** Typed evidence payloads, stage-local diagnostics, strategy artifacts, and rollback/gate records keep heterogeneous modules aligned while preserving each track’s claim boundary.
- **Evidence-bounded anchors:** Structure, Evidence Admission, Evolution, and Deployment each contribute calibrated evidence rather than a single aggregate leaderboard.
- **Traceable improvement:** Representative diagnostics, rejected directions, and gate decisions are preserved as auditable development artifacts.

The next practical step is to turn these artifact families into operational infrastructure for telemetry, privacy controls, editability, rollback, and reproducible ablation.

Research Outlook

The four lifecycle gaps identified in Section 1—state, source, governance, and deployment—are partially addressed here, but none is closed by a single component. Closing them requires co-design across algorithms, shared evaluation contracts, and product-level validation. Five directions are most immediate.

Causal memory attribution. The open problem is determining the minimal evidence set that entails a given answer and verifying that no intermediate stage introduced unsupported content. This specializes faithfulness verification to multi-granularity stores where the same fact may exist as an L0 atomic fact, an L1 summary, and an L2 profile entry. The product target is user-facing provenance and operator debugging: when a user asks “why did you say that?”, the system should trace the answer to specific memories and retrieval decisions.

Propagation-complete forgetting. Current forget-guard mechanisms do not guarantee that a revoked fact ceases to influence downstream summaries, profiles, or cached retrievals. The challenge is to invalidate or re-derive all dependent representations when evidence is deleted—this intersects with machine unlearning [3] but operates over structured memory, not parametric weights. The product requirement is demonstrable compliance through stage-local diagnostic traces.

Federated memory composition across devices. The research problem is maintaining a coherent user model when phones, cars, wearables, and homes contribute evidence under different retention, privacy, and sharing policies without one centralized store. This extends federated learning ideas to structured memory state and connects to policy-aware synchronization, conflict resolution, and trust-boundary enforcement.

Diagnostic-driven evolution across deployments. Scaling governed memory evolution from offline benchmark stages to production traffic requires online monitoring without ground-truth labels, automated hypothesis generation from failure clusters, and guarded incremental deployment with category-sensitive rollback safeguards. These problems connect to safe reinforcement learning and automated ML pipeline management, specialized to memory systems where improving one segment can silently regress another.

Standardized memory contracts. The community lacks a shared interface contract—analogue to function-calling schemas or tool-use protocols—for cross-system ablation, portable benchmarks, and composable evolution across heterogeneous memory architectures. The research question is what minimal contract can support diverse systems while staying constrained enough for meaningful comparison and governed interoperability.

Long-term convergence. The directions above converge on governed, multi-device, self-diagnosing assistant-memory infrastructure. The central open problem is making the contract *expressive* across sources and modalities, *auditable* from research through production, and *transferable* across cloud, edge, and local substrates. No single module in this report achieves all three; the four lifecycle roles instead break the target into testable pieces. Long term, deployed observations should generate hypotheses, controlled tests should validate them under the shared contract, accepted changes should ship behind gates, and deployment outcomes should feed the next cycle of structure refinement, source expansion, and governance improvement.

References

- [1] Yuanchen Bei, Tianxin Wei, Xuying Ning, Yanjun Zhao, Zhining Liu, Xiao Lin, Yada Zhu, Hendrik Hamann, Jingrui He, and Hanghang Tong. Mem-Gallery: Benchmarking multimodal long-term conversational memory for MLLM agents. *arXiv preprint arXiv:2601.03515*, 2026.
- [2] Varun Pratap Bhardwaj. SuperLocalMemory v3.3: The living brain—biologically-inspired forgetting, cognitive quantization, and multi-channel retrieval for zero-LLM agent memory systems. *arXiv preprint arXiv:2604.04514*, 2026.
- [3] Lucas Bourtole, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning. In *IEEE Symposium on Security and Privacy*, pages 141–159, 2021.
- [4] Hanxiang Chao, Yihan Bai, Rui Sheng, Tianle Li, and Yushi Sun. STALE: Can LLM agents know when their memories are no longer valid? *arXiv preprint arXiv:2605.06527*, 2026.
- [5] Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. HarnessX: A composable, adaptive, and evolvable agent harness foundry. *arXiv preprint arXiv:2606.14249*, 2026.
- [6] Zhikai Chen, Jialiang Gu, Junyu Yin, Xianxuan Long, Shenglai Zeng, Xiaoze Liu, Kai Guo, Keren Zhou, and Jiliang Tang. Exploring cross-scenario generality of agentic memory systems: Diagnostics and a strong baseline. *arXiv preprint arXiv:2606.04315*, 2026.
- [7] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [8] Daniel Comney. When generic prompt improvements hurt: Evaluation-driven iteration for LLM applications. *arXiv preprint arXiv:2601.22025*, 2026.
- [9] Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, HuaJun Chen, and Ningyu Zhang. LightMem: Lightweight and efficient memory-augmented generation. *arXiv preprint arXiv:2510.18866*, 2025.
- [10] Tianxiang Fei, Mingyang Song, Mao Zheng, and Xiang Yu. Memory beyond recall: A dual-process cognitive memory system for self-evolving LLM agents. *arXiv preprint arXiv:2606.09483*, 2026.
- [11] Junyu Feng, Binxiao Xu, Jiayi Chen, Mengyu Dai, Cenyang Wu, Haodong Li, Bohan Zeng, Yunliu Xie, Hao Liang, Ming Lu, and Wentao Zhang. M2A: Multimodal memory agent with dual-layer hybrid memory for long-term personalized interactions. *arXiv preprint arXiv:2602.07624*, 2026.
- [12] Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland. MemoryArena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv preprint arXiv:2602.16313*, 2026.
- [13] Weiquan Huang, Zixuan Wang, Hehai Lin, Sudong Wang, Bo Xu, Qian Li, Beier Zhu, Linyi Yang, and Chengwei Qin. AMA: Adaptive memory via multi-agent collaboration. *arXiv preprint arXiv:2601.20352*, 2026.
- [14] Zhengjun Huang, Wenxuan Liu, Zhoujin Tian, Wei Chen, Junle Chen, Yuqian Wu, Fangyuan Zhang, Qintian Guo, and Xiaofang Zhou. M³Exam: Benchmarking multimodal memory for realistic user-agent interactions. *arXiv preprint arXiv:2606.07402*, 2026.
- [15] Bowen Jiang, Yuan Yuan, Maohao Shen, Zhuoqun Hao, Zhangchen Xu, Zichen Chen, Ziyi Liu, Anvesh Rao Vijjini, Jiashu He, Hanchao Yu, Radha Poovendran, Gregory Wornell, Lyle Ungar, Dan Roth, Sihao Chen, and Camillo Jose Taylor. PersonaMem-V2: Towards personalized intelligence via learning implicit user personas and agentic memory. *arXiv preprint arXiv:2512.06688*, 2025.
- [16] Chingkwun Lam, Jiaxin Li, Lingfei Zhang, and Kuo Zhao. Governing evolving memory in LLM agents: Risks, mechanisms, and the stability and safety governed memory (SSGM) framework. *arXiv preprint arXiv:2603.11768*, 2026.
- [17] Zhuofeng Li, Haoxiang Zhang, Cong Wei, Pan Lu, Ping Nie, Yi Lu, Yuyang Bai, Shangbin Feng, Hangxiao Zhu, Ming Zhong, Yuyu Zhang, Jianwen Xie, Yejin Choi, James Zou, Jiawei Han, Wenhui Chen, Jimmy Lin, Dongfu Jiang, and Yu Zhang. Beyond semantic similarity: Rethinking retrieval for agentic search via direct corpus interaction. *arXiv preprint arXiv:2605.05242*, 2026.

- [18] Zehao Lin, Xixuan Hao, Renyu Fu, Shaobo Cui, Kai Chen, Chunyu Li, Zhiyu Li, and Feiyu Xiong. A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle. *arXiv preprint arXiv:2604.16548*, 2026.
- [19] Chengzhi Liu, Yuzhe Yang, Sophia Xiao Pu, Yepeng Liu, Lin Long, Yichen Guo, Nuo Chen, Zhaotian Weng, Elena Kochkina, Simerjot Kaur, Charese Smiley, Xiaomo Liu, James Zou, Sheng Liu, Yuheng Bu, Songyou Peng, and Xin Eric Wang. WorldMemArena: Evaluating multimodal agent memory through action-world interaction. *arXiv preprint arXiv:2605.29341*, 2026.
- [20] Jiaqi Liu, Xinyu Ye, Peng Xia, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. EvolveMem: Self-evolving memory architecture via AutoResearch for LLM agents. *arXiv preprint arXiv:2605.13941*, 2026.
- [21] Junming Liu, Yifei Sun, Weihua Cheng, Haodong Lei, Yirong Chen, Licheng Wen, Xuemeng Yang, Daocheng Fu, Pinlong Cai, Nianchen Deng, Yi Yu, Shuyue Hu, Botian Shi, and Ding Wang. MemVerse: Multimodal memory for lifelong learning agents. *arXiv preprint arXiv:2512.03627*, 2025. v2, updated 2026.
- [22] Junming Liu, Yifei Sun, Weihua Cheng, Haodong Lei, Yuqi Li, Yirong Chen, and Ding Wang. Hierarchical memory orchestration for personalized persistent agents. *arXiv preprint arXiv:2604.01670*, 2026.
- [23] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL <https://arxiv.org/abs/2402.17753>.
- [24] Stefan Miteski. Memory as metabolism: A design for companion knowledge systems. *arXiv preprint arXiv:2604.12034*, 2026.
- [25] Andy Nguyen, Danh Doan, Hoang Pham, Bao Ha, Dat Pham, Linh Nguyen, Hieu Nguyen, Thien Nguyen, Cuong Do, Phat Nguyen, and Toan Nguyen. ByteRover: Agent-native memory through LLM-curated hierarchical context. *arXiv preprint arXiv:2604.01599*, 2026.
- [26] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [27] Jingyang Qiao, Weicheng Meng, Yu Cheng, Zhihang Lin, Zhizhong Zhang, Xin Tan, Jingyu Gong, Kun Shao, and Yuan Xie. Memory intelligence agent. *arXiv preprint arXiv:2604.04503*, 2026. URL <https://arxiv.org/abs/2604.04503>.
- [28] Xiyu Ren, Zhaowei Wang, Yiming Du, Zhongwei Xie, Chi Liu, Xinlin Yang, Haoyue Feng, Wenjun Pan, Tianshi Zheng, Baixuan Xu, Zhengnan Li, Yangqiu Song, Ginny Wong, and Simon See. MemLens: Benchmarking multimodal long-term memory in large vision-language models. *arXiv preprint arXiv:2605.14906*, 2026.
- [29] Pavan C Shekar, Abhishek H S, and Aswanth Krishnan. GitOfThoughts: Version-controlled reasoning and agent memory you can replay, diff, and merge. *arXiv preprint arXiv:2606.14470*, 2026.
- [30] Baris Simsek. When to forget: A memory governance primitive. *arXiv preprint arXiv:2604.12007*, 2026.
- [31] Taranjeet Singh. Introducing the token-efficient memory algorithm. *Mem0 Research Blog*, apr 2026. URL <https://mem0.ai/blog/mem0-the-token-efficient-memory-algorithm>.
- [32] Mohammad Tavakoli et al. Beyond a million tokens: Benchmarking and enhancing long-term memory in LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://arxiv.org/abs/2510.27246>.
- [33] Md Nayem Uddin, Kumar Shubham, Eduardo Blanco, Chitta Baral, and Gengyu Wang. From recall to forgetting: Benchmarking long-term memory for personalized agents. *arXiv preprint arXiv:2604.20006*, 2026.
- [34] Shu Wang, Edwin Yu, Oscar Love, Tom Zhang, Tom Wong, Steve Scargall, and Charles Fan. MemMachine: A ground-truth-preserving memory system for personalized AI agents. *arXiv preprint arXiv:2604.04853*, 2026.
- [35] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2410.10813>.
- [36] Di Wu, Zixiang Ji, Asmi Kawatkar, Bryan Kwan, Jia-Chen Gu, Nanyun Peng, and Kai-Wei Chang. LongMemEval-V2: Evaluating long-term agent memory toward experienced colleagues. *arXiv preprint arXiv:2605.12493*, 2026.
- [37] Junde Wu, Minhao Hu, Jiayuan Zhu, Jiazhen Pan, Yuyuan Liu, Min Xu, and Yueming Jin. Git context controller: Manage the context of LLM-based agents like Git. *arXiv preprint arXiv:2508.00031*, 2025. v2, updated 2026.

- [38] Menglin Xia, Xuchao Zhang, Shantanu Dixit, Paramaguru Harimurugan, Rujia Wang, Victor Ruhle, Robert Sim, Chetan Bansal, and Saravan Rajmohan. Memora: A harmonic memory representation balancing abstraction and specificity. *arXiv preprint arXiv:2602.03315*, 2026.
- [39] Jundong Xu, Qingchuan Li, Jiaying Wu, Yihuai Lan, Shuyue Stella Li, Huichi Zhou, Bowen Jiang, Lei Wang, Jun Wang, Anh Tuan Luu, Caiming Xiong, Hae Won Park, Bryan Hooi, and Zhiyuan Hu. EvoArena: Tracking memory evolution for robust LLM agents in dynamic environments. *arXiv preprint arXiv:2606.13681*, 2026.
- [40] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schütze, Volker Tresp, and Yunpu Ma. Memory-R1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.
- [41] Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, Zhimeng Jiang, Shaochen Zhong, and Xia Hu. Data-centric artificial intelligence: A survey. *ACM Computing Surveys*, 57(5):1–42, 2025. doi: 10.1145/3711118.
- [42] Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. MemSkill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*, 2026.
- [43] Jiawen Zhang, Kejia Chen, Jiachen Ma, Yangfan Hu, Lipeng He, Yechao Zhang, Jian Liu, Xiaohu Yang, Tianwei Zhang, and Ruoxi Jia. Beyond similarity: Trustworthy memory search for personal AI agents. *arXiv preprint arXiv:2606.06054*, 2026.
- [44] Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. Externalization in LLM agents: A unified review of memory, skills, protocols and harness engineering. *arXiv preprint arXiv:2604.08224*, 2026.
- [45] Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, and Jun Wang. Memento: Fine-tuning LLM agents without fine-tuning LLMs. *arXiv preprint arXiv:2508.16153*, 2025. URL <https://arxiv.org/abs/2508.16153>.

Contributions and Acknowledgments

Core Contributors.

- Xule Liu*
- Hanlin Teng*
- Chao Li*
- Yanan Ni
- Kun Shao[†]
- Jian Luan[†]

Contributors.

- Shuo Lu
- Audrey Wang
- Yijun Liu
- Yunfei Wang
- Xiaofeng Li
- Xian Yi
- Yuanfa Li
- Kang Zhao
- Jian Liang
- Yuxuan Chen
- Jinyuan Chen
- Heng Qu

* Equal contribution. [†] Corresponding author.

Appendix

The appendix collects notation, extended module details, artifacts, and evaluation protocol details referenced in the main text. It is organized as follows:

- **Section A** — Notation and symbols (principal symbols by lifecycle role)
- **Section B** — MemStack extended details (terminology, interfaces, retrieval, QA prompt, design principles, optional hooks)
- **Section C** — Evidence Admission extended artifacts (MemFuse ingestion and session graph)
- **Section D** — MemSense extended artifacts (IKB passes, answer algorithm, progress)
- **Section E** — D²ACCI extended artifacts (iteration rounds, verification infrastructure, case studies)
- **Section F** — E²MEND evolution artifacts (loop algorithm, strategy schema, gate/critic rules, UCB1)
- **Section G** — LiteMem extended artifacts (schema, hierarchy, ranking equations, lifecycle algorithm)
- **Section H** — Evaluation protocol details (claim checks, root-cause summary, cost)

A Notation and Symbols

Table 13 summarizes the principal symbols used throughout the report.

Table 13 Principal notation.

Symbol	Introduced	Meaning
<i>Overview / lifecycle contract (Section 3)</i>		
q_t	§3	User request at turn t
O_t	§3	Typed source-observation pool available at serving time
M_t	§3	Memory state before serving at turn t
E_{q_t}	§3	Evidence selected for request q_t
C_{q_t}	§3	Bounded context assembled for generation
a_t	§3	Generated answer at turn t
M_{t+1}	§3	Memory state after improvement-loop update
<i>Structure / MemStack (Section 4)</i>		
$L_{0,t}, L_{1,t}, L_{2,t}, SM_t$	§4	Layerwise realization of M_t : atomic facts, session summaries, profile constraints, and session memory
o_t	§4	Admitted observation at turn t
q_t	§4	User request at turn t (same as overview q_t)
W	§4	Layer-aware write operator
$R_{\text{sem}}, R_{\text{lex}}, R_{\text{exp}}$	§4	Semantic, lexical, and expanded retrieval channels
$\hat{E}_{q_t}$	§4	Combined retrieval candidates before budgeting
A	§4	Assembly function
B	§4	Context budget
U	§4	Update operator
c_t	§4	Correction arriving after generation
<i>Expansion / Source Layer (Section 5)</i>		
O	§5	Typed observation universe (text, images, device events)
G	§5	Cross-device memory graph
K	§5	Image Knowledge Base (IKB)
λ	§5	Traceability weight
e	§5	Individual candidate evidence item
$\text{rel}(e, q_t)$	§5	Relevance of evidence e to query q_t
$\text{tok}(e)$	§5	Token cost of evidence e
$\text{prov}(e)$	§5	Provenance score of evidence e
Rel_j	§5	Related L0 facts for image x_j in IKB (avoids overloading R)
e_t	§5	Incoming event at turn t
N_t	§5	Retrieved neighborhood for fusion
<i>Deployment / LiteMem (Section 7)</i>		
$\text{Score}(m, q_t, t)$	§7	Retrieval scoring function
$\lambda_1 - \lambda_4$	§7	Component weights (lexical, importance, recency, access)
$\text{Imp}_t(m)$	§7	Importance with decay at turn t
Δt	§7	Interval since record was last accessed (retrieval recency)
Δt_m	§7	Interval since importance was last assigned (organization decay)
τ, τ_m	§7	Recency decay timescales (τ short, τ_m long)
η, ρ	§7	Access-boost and skip-penalty coefficients

B MemStack Extended Details

This section provides the MemStack details used to interpret and reproduce the reported runs: terminology, typed interfaces, retrieval formula, context-budget policy, design principles, and the optional procedural-hook boundary.

Terminology. Table 14 defines the terms used across the framework. These definitions are shared by all modules; later sections reference them without redefining them.

Table 14 Terminology.

Term	Definition
Memory item	A typed stored unit in MemStack, including L0 facts, L1 summaries, L2 profile entries, or optional ProcedureEntry payloads.
Evidence	Source observations or memory items that support a particular answer, diagnosis, or strategy decision, together with provenance and trace identifiers.
Policy	The operational behavior governing ingestion, retrieval, filtering, context construction, and generation support.
Strategy artifact	A declarative, versioned policy artifact consumed by E ² MEND; it changes parameters, prompts, and routing rules without modifying framework code.
Strategy space	The schema-constrained set of admissible strategy artifacts and candidate mutations searched by E ² MEND.
Human hypothesis	A D ² ACCI iteration proposal written by an engineer to explain a failure mode and predict how a bounded intervention should affect metrics.
Harness	A fixed evaluation environment containing dataset slice, prompts, judge, scoring script, random seeds, and acceptance criteria.
Provider	An implementation backend for storage, retrieval, model calling, judging, or source ingestion behind a stable interface.
Optional procedural guidance	A typed trigger-procedure payload attached to MemStack context assembly when enabled.
FusionSession	A MemFuse grouping of cross-device events with three transient zones whose accepted edges are persisted after zone eviction.
MemoryPack	A MemFuse output payload that summarizes an activity-level fused node while retaining links to atomic events, devices, timestamps, and causal/semantic edges.
Diagnostic trace	A JSONL-style per-question artifact recording retrieved candidates, filters, ranks, context packing, answer evidence, and failure attribution.
Gate	A deterministic or critic-assisted acceptance check that rejects schema-violating, regressive, or non-replayable strategy changes before serving-strategy reload.

Typed interfaces. Table 15 specifies the five payload types that cross module boundaries. Each row states the required fields, producer-consumer pair, current implementation status, and failure-handling policy.

Table 15 Interface excerpt.

Payload	Required fields	Producer → consumer	Implemented / evidence status	Failure handling
FusedEvent	id, source_event_ids, timestamp, device_ids, edge_type, provenance, confidence	MemFuse → MemStack	Prototype path used by internal MemFuseBench evidence	Keep atomic events if fusion is low-confidence or contradictory
PerceptionFact	fact_id, image_id, session/date, category/name, caption, confidence, source_ids	MemSense → MemStack	Used by module-level Mem-Gallery run	Fall back to image id when names/categories are uncertain
ProcedureEntry	entry_id, trigger, procedure, constraints, validation, status	MemStack → Context Assembly	Design-only; internal to MemStack	Operational guidance only
DiagnosticSignal	question_id, retrieved_ids, context_ids, answer_evidence, stage_label, error_label	MemStack → E ² MEND	Used when evidence annotations exist	Mark stage label uncertain when source ids are incomplete
StrategyArtifact	artifact_id, schema_version, mutation_paths, guardrails, gate_record, rollback_key	E ² MEND → MemStack	Used by the staged E ² MEND LoCoMo run	Reject if schema or non-regression gates fail; restore on rollback

Safety and privacy gaps. Table 16 maps known risk surfaces to the mechanism that partially addresses them and the residual gap that remains outside the scope of module-level evidence.

Table 16 Safety/privacy gaps.

Risk	Mechanism	Evidence	Gap
Stale/harmful memory	Provenance, forget/update constraints, invalidation	con- MemStack schema + traces	User-facing editing policy pending
Cross-device linkage	Source provenance, visibility data, typed payloads	meta- MemFuse interface design	Consent/ACL are platform-level
Strategy drift	Schema-constrained artifacts, checks, rollback	gate E ² MEND dev-slice trace	Online evolution without labels
Conflict/mis-fusion	Atomic-event provenance, fused-edges	Preliminary MemFuseBench	Conflict arbitration unresolved
Local exposure	Markdown/Git boundary, local-file-ecution	LiteMem transfer run	OS permissions, encryption

Capability map and interfaces. Tables 17 and 18 trace capabilities from lifecycle stages to modules, then detail the inter-module triggers and payloads.

Table 17 Capability map.

Stage	Core Capability	Module
Sources	Dialogue, device events, visual evidence	MemFuse + MemSense
Admission and ingestion	Source normalization, segmentation, extraction, consolidation	MemSense + MemFuse + MemStack
Representation	L0/L1/L2 facts, optional procedural hooks, fused nodes	MemStack
Retrieval	Hybrid search, rerank, graph expansion	MemStack
Context assembly	Budgeted packing, deduplication, safety constraints	MemStack
Feedback & update	User correction, decay, invalidation, Dreaming/idle consolidation	MemStack + D ² ACCI
Governed evolution	Diagnostics, strategy search, gated strategy reload	D ² ACCI + E ² MEND
Deployment	Repository-native transfer feasibility, privacy-sensitive mode	LiteMem

Table 18 Interfaces and triggers.

Source	Target	Interface	Trigger	Payload
MemFuse	MemStack	FusedEvent []	Device event processed	Causal events + device provenance
MemSense	MemStack	PerceptionFact []	Perception pipeline completed	Structured visual facts + confidence
MemStack †	Context Assembly	ProcedureEntry []	Intent / pattern match	Optional user-specific procedural guidance
MemStack	E ² MEND	DiagnosticSignal	Diagnostic logging	Coverage/rank/error traces
E ² MEND	MemStack	StrategyArtifact	Gate approval	Updated declarative config

† The optional procedural hook is internal to MemStack; it is design-only and listed separately because it has a distinct output interface to Context Assembly.

Three-channel retrieval. Given query q_t and memory store M_t , the runtime combines semantic vector search, lexical BM25 matching, and LLM-generated subquery expansion. Let $L_c(q_t)$ be the ranked list returned by channel $c \in \{\text{vec}, \text{bm25}, \text{subq}\}$ and $\text{rank}_c(m)$ the one-based rank of memory m in that list. Candidate lists are fused by weighted Reciprocal Rank Fusion,

$$\text{RRF}(m) = \sum_{c \in \{\text{vec}, \text{bm25}, \text{subq}\}} \frac{w_c \mathbf{1}[m \in L_c(q_t)]}{\kappa + \text{rank}_c(m)}. \quad (10)$$

Unless overridden by a benchmark adapter, κ follows the common RRF default of 60, channel weights start uniform, and the final candidate pool is truncated after deduplication and optional reranking. LLM subquery expansion is bounded by a fixed configuration value (three to five subqueries in the reported runs) and must preserve explicit entities, dates, and negations from the original query. The exact prompt is treated as a versioned strategy artifact when evolved by E²MEND; changing it inside framework code is a D²ACCI-level change. CombMNZ and Borda are retained as ablation alternatives but are not used for the main reported path. Table 19 lists the retrieval parameters exposed as configuration together with their claim boundaries.

Table 19 Retrieval parameters.

Parameter	Role	Claim boundary
κ	Smooths RRF rank contribution; default 60 unless adapter overrides	Not tuned per question
$w_{\text{vec}}, w_{\text{bm25}}, w_{\text{subq}}$	Channel weights for semantic, lexical, and subquery paths	Uniform baseline; evolvable only as declared strategy
<code>subquery.max_n</code>	Caps LLM query decompositions	Small bounded value; prompt changes require artifact versioning
<code>rerank.top_n</code>	Limits expensive reranking after RRF	Configuration knob, recorded in trace
<code>dedup.threshold</code>	Prevents near-duplicate L0/L1 evidence from consuming budget	Validated by paired regression gates

Context budget assembly. Context assembly reserves profile evidence, treats forget/update constraints as high-priority governance constraints, and fills remaining context by fused rank after deduplication. A typical packing order is shown in Table 20. The percentages are defaults used to explain the policy, not universal constants: each run records the realized token counts so a regression can be attributed to retrieval, filtering, or packing.

Table 20 Context budget.

Slot	Default policy	Overflow behavior
System/task instructions	Fixed prefix	Not counted as memory evidence
Forget/update constraints	Mandatory, budget-exempt in priority	Reduce lower-ranked L0/L1 before dropping constraints
L2 profile / style	Reserved high-priority slice (typically 10–20%)	Summarize profile, keep provenance pointer
SM current session	Recent unresolved turns and constraints	Window by recency and unresolved state
L0/L1 evidence	Pack by fused rank after deduplication	Emit overflow trace with dropped ids
Source payloads	IKB / <code>FusedEvent</code> evidence when routed	Exempt from broad cosine pruning; still token-bounded

Assembly and adapters. Benchmark-specific logic remains in adapters, while ingestion, storage, retrieval, assembly, and diagnostics stay in the shared kernel.

Representative QA system prompt. This prompt governs answer generation in the QA evaluation path. At the generation stage, it instantiates the audit contract by requiring explicit evidence enumeration from memory layers, provenance-chain resolution, temporal grounding, contradiction arbitration via correction history, and sufficiency verification before committing to a final answer. The structured step format allows diagnostic traces to attribute failures to specific reasoning stages rather than opaque generation.

`prompts/qa_system.txt`

```
You are an evidence-governed memory assistant. You answer
questions by retrieving and reasoning over structured
personal memory organized into layers: L0 (atomic facts),
L1 (session/topic summaries), L2 (stable profile and
corrections), and SM (current session context).
Today's date is {today_date}.
```

```
# EVIDENCE FIDELITY REQUIREMENTS
```

1. Preserve all named entities verbatim -- use full identifiers (e.g. "Amy's colleague Rob"), never generic references
2. Retain exact quantities: numbers, prices, dates, times, percentages, frequencies
3. Maintain temporal specificity -- "every Tuesday and Thursday", not "twice a week"
4. When multiple records describe similar events, use timestamps and layer metadata to distinguish them

5. Perform inference only when evidence from multiple layers strongly supports the connection

STRUCTURED REASONING PATH

Step 1: EVIDENCE CANDIDATES

Enumerate all memory records (L0 facts, L1 summaries, L2 profile entries, SM context) that could relate to the question. Include records with unresolved relative dates -- do not filter prematurely.

Step 2: LAYER-AWARE DETAIL EXTRACTION

For each candidate, extract: source layer, record timestamp, entity names, quantities, and provenance links to other records.

Step 3: PROVENANCE CHAIN RESOLUTION

Trace connections across layers and records:

- L0-to-L2 promotion: if an atomic fact establishes a stable attribute, link to the profile entry
- Cross-entity inference: properties of linked entities (family members, colleagues) may imply attributes
- Collective references: resolve "they/we/together" to specific entities via session context

Step 4: TEMPORAL GROUNDING

Resolve relative time expressions using record metadata:

- "yesterday" from a record dated 2023-08-25 means 2023-08-24
- Use [recorded:] and [updated:] timestamps for ordering; [previously:] marks superseded versions
- Never assume today's date unless the record explicitly states "today" or "just now"
- Only use dates explicitly present in retrieved evidence for temporal arithmetic

Step 5: CONTRADICTION ARBITRATION

When records conflict on the same attribute, apply the correction-history rule: the most recently updated L2 entry or the latest L0 observation takes precedence. Flag the conflict source in your reasoning.

Step 6: EVIDENCE-QUESTION ALIGNMENT

- Re-read the question and identify the specific event, time, or context it targets
- For each detail in your answer, verify it originates from the SAME event/context the question asks about
- When multiple similar records exist, match question constraints to the correct record explicitly

Step 7: SUFFICIENCY VERIFICATION

If no single record contains the answer, synthesize across layers. Commit to the strongest supported inference -- only respond "insufficient evidence" when no record is even tangentially related.

FINAL ANSWER

State the answer directly and concisely first, then add supporting provenance. Do not hedge when evidence is available.

FINAL ANSWER: <your answer>

Core design principles. The following principles were refined through the D²ACCI development history and guide module-level design decisions. Each principle is an engineering constraint supported by the available paired evaluations; violations surface as regressions in the corresponding module’s evaluation. Table 21 summarizes each principle together with its concrete embodiment in the system.

Table 21 Design principles.

Principle	Meaning	Embodiment
Typed boundaries	Modules exchange explicit payloads, not internal state	FusedEvent / PerceptionFact / DiagnosticSignal
Decoupled evolution	Stable framework locked, variable strategies declarative	Framework/Strategy separation
Progressive disclosure	On-demand expansion, no full-history preload	L0/L1/L2 + optional hooks + lazy cascade
Causal multi-source	Causal evidence complements semantic similarity	MemFuse dual-layer graph
Evidence-governed improvement	Every change needs paired evidence and bounded regressions	Diagnostic trace + gate decision
Edge transferability	Core memory principles can be simplified for local deployment	LiteMem transfer evidence

B.1 Optional procedural hooks inside MemStack

MemStack can store a compact procedural guidance entry when repeated user feedback shows that factual recall alone is not enough. The hook was introduced during D²ACCI iterations after repeated errors exposed a distinct failure pattern: the system could remember the relevant facts but still fail to adapt its response procedure to a user’s recurring feedback, routines, or conversational expectations. These failures motivate a memory object that stores *how the assistant should interact with this user in a recurring situation*, not only what the user said. Unlike general-purpose agent skill memory or tool-skill libraries, the hook is scoped to personalized conversational behavior and remains internal to MemStack.

A procedural entry is represented as a compact contract:

$$\text{ProcedureEntry} = (\text{trig}, p, c, v), \quad (11)$$

where *trig* is the trigger condition, *p* is an ordered conversational procedure, *c* is the constraint set, and *v* is the validation rule applied before the final answer or experiment report. Triggers decide when the procedural entry is relevant, procedures encode reusable user-specific response steps, constraints record what must not be violated, and validation rules convert the entry into an executable checklist. This differs from preference memory: a preference states what the user wants, while the procedural entry states how the assistant should reliably respond to that user’s recurring task or situation.

At runtime, procedural entries are retrieved only after the topic, entity, temporal, and confidence gates pass. The gated entry is injected as operational guidance only, keeping procedural rules separate from answer content so diagnostics can distinguish factual failures from interaction-procedure failures. The hook is therefore framed as a design outcome of human-guided diagnostic iteration, not a public benchmark track. Limited internal diagnostic cases are used only to check the mechanism; large-scale cross-domain evaluation and ablation are left to future work.

C Extended Evidence Admission Artifacts

This section adds implementation-level MemFuse diagrams and FusionSession mechanics to the Evidence Admission chapter. Figure 10 illustrates device-specific payload normalization before fusion, so later retrieval can operate over fused nodes rather than raw sensor formats alone. Figure 11 motivates the dual representation in MemFuse: atomic nodes preserve provenance, while fused nodes provide the activity-level memory used by downstream retrieval.

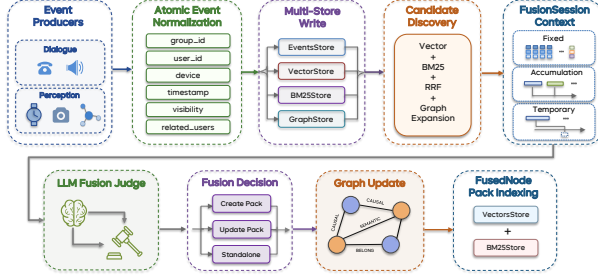


Figure 10 MemFuse ingestion path.

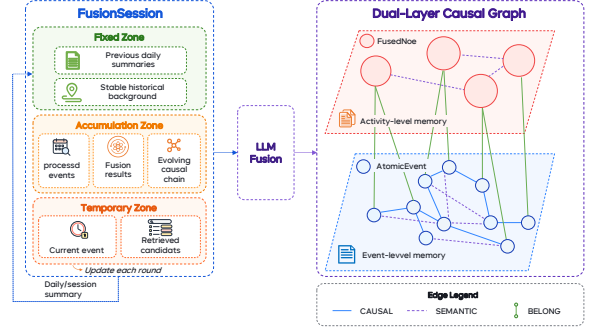


Figure 11 MemFuse session graph.

D Extended MemSense Artifacts

This section adds the IKB construction pipeline (Table 22), the IKB-first answer algorithm (Algorithm 1), and implementation status with remaining gaps (Table 23) for the multimodal-memory branch. The five-pass construction in Table 22 allows Algorithm 1 to prefer symbolic image knowledge over top- k visual retrieval.

Table 22 IKB build passes.

Pass	Operation	Failure Addressed
1	Per-image conversation-name extraction with preceding turn context	Resolves references such as “another one” or assistant-provided species names
2	Session-level batch naming for unnamed images	Recovers images whose local turn lacks enough context
3	Synonym / variant normalization	Merges names such as “frangipani” and “plumeria” when they denote the same object
4	Similarity-check split for large groups	Over-merged categories that inflate counts
5	Registry backfill	Missing fallback IKB entries for registered images

The staged build directly affects visual counting and enumeration questions, where a missing registry entry or over-merged synonym can change the final answer.

Algorithm 1 specifies how the constructed IKB is used at answer time. The algorithm functions as a routing rule: standard text memory remains available, but VR, VS, and TTL cases receive deterministic image-knowledge injections before generation.

This algorithm links construction to evaluation: the remaining errors in Table 23 are mostly trigger, normalization, and VLM-override issues, not missing support for multimodal retrieval itself.

Table 23 summarizes implementation status so that remaining engineering gaps are not confused with completed benchmark claims.

The table pairs each implemented component with its remaining gap, keeping the appendix evidence conservative: it documents what is already implemented in the branch and what still needs robustness work.

E Extended D²ACCI Artifacts

This section summarizes the D²ACCI evidence trail: paired comparison sets, failure-case traces, root-cause labels, patch diffs, statistical reports, experiment logs, and representative accepted/rejected rounds. It provides methodology evidence, not a benchmark result.

Diagnostic trace and constrained optimization. Let $\mathcal{S} = \{s_i\}_{i=1}^N$ be an evaluation set and $\mathcal{P}_\theta$ a memory configuration. For each sample, the shared kernel emits a diagnostic trace

$$d_i = (e_i^{\text{raw}}, r_i^{\text{top-k}}, f_i^{\text{filter}}, c_i^{\text{ctx}}, g_i^{\text{gen}}, j_i), \quad (12)$$

covering ingestion, retrieval, filtering, context assembly, generation, and judging. The objective is constrained improvement:

$$\theta^* = \arg \max_{\theta \in \Theta} \text{Acc}(\theta) \quad \text{s.t.} \quad \forall c \in \mathcal{C} : \Delta_c(\theta, \theta_{\text{base}}) \geq -\epsilon_c, \quad (13)$$

Algorithm 1: IKB-first answer path.**Input:** Question q , optional question image x_q , mem0 store M , RAGAnything store R , IKB K **Output:** Answer a

- 1 Classify q into intent and retrieval strategy
 - 2 Retrieve text memories from M using the selected strategy
 - 3 Run session filter/rerank over Session Memory candidates
 - 4 Retrieve graph chunks and image candidates from R using text–text, text–image, and image–image paths
 - 5 Fuse L0/L1/L2, session context, graph chunks, entities, relations, and reasoning paths
 - 6 **if** q is VR counting and K matches a category **then**
 - 7 | inject count and image ID evidence from K when available; constrain output format to number only
 - 8 **end**
 - 9 **else if** q is VS enumeration or time-filtered selection **then**
 - 10 | inject candidate image list from K using category/session/temporal indexes
 - 11 **end**
 - 12 **else if** q is TTL with question image **then**
 - 13 | match x_q against conversation images; inject top conversation names from K
 - 14 **end**
 - 15 Generate final answer with text context and at most eight retrieved images
-

Table 23 Multimodal progress.

Component	Implementation Status	Open Gap
IKB build	Five-pass construction, temporal index, related-fact binding	Synonym normalization can over-merge fine-grained categories
VR counting	IKB count query with format override and registry cross-check	Fuzzy aggregation can inflate counts; needs stricter category confidence
VS enumeration	IKB-based candidate-list path exists for “find all” patterns	Triggering remains sensitive; some VS questions still rely on image-vector top- k
TTL naming	Candidate conversation-name injection exists; image-match helper exists	Deterministic image matching is not yet first-class in the main TTL path
RAGAnything retrieval	Text–text, text–image, image–image, graph PPR, reasoning paths	Cross-session visual recall can truncate before IKB enumeration is applied
Answer formatting	Category-specific format instructions and JSON/no-JSON routing	VLM can override textual constraints for visually ambiguous images
Knowledge filter	Optional evidence filter and AR (Association/Reasoning) post-processing	Disabled by default; AR is a local error-analysis tag for associative or reasoning hallucinations, not a fourth visual category alongside VR/VS/TTL

where $\mathcal{C}$ is the category set and ϵ_c bounds acceptable regression.

Evidence-preservation ladder. Stage attribution follows the earliest point where ground-truth evidence e_i^* is lost:

$$\text{FailType}(s_i) = \begin{cases} \text{ingestion_gap} & \text{if } e_i^* \notin \text{MemStore}(\theta) \\ \text{retrieval_gap} & \text{if } e_i^* \in \text{MemStore} \setminus r_i^{\text{top-k}} \\ \text{kf_filtered} & \text{if } e_i^* \in r_i^{\text{top-k}} \setminus c_i^{\text{ctx}} \\ \text{generation_error} & \text{if } e_i^* \in c_i^{\text{ctx}} \text{ but } J(\hat{y}_i, y_i^*) = 0. \end{cases} \quad (14)$$

The **kf_filtered** label refers to evidence removed by the knowledge filter during context assembly.

Representative iteration rounds. Table 24 provides the roadmap for this appendix: it lists which iterations are discussed later, why each change was accepted or rejected, and how the diagnostic labels connect the raw traces to the reported score movement.

The table highlights that most accepted gains came from fixing a localized failure surface rather than adding broad prompt complexity; the rejected rows are included because they became negative priors for later automated search.

Table 24 D²ACCI rounds.

Round	Hypothesis / intervention	Root cause	Evidence gate	Score	Δ	Decision
t_0	Mem0-style pipeline	—	establishes aligned baseline	83.07	—	baseline
t_1	CoT 7-step + time-aware prompt	generation	paired judge gain without category regression	88.83	+5.76	accept
t_2	Ingestion ordering + update fix	ingestion	missing-evidence traces recovered	90.70	+1.87	accept
t_3	Dedup threshold 0.75	over-merge	regression exceeds non-regression gate	88.25	−2.45	revert
t_4	Threshold 0.92 + topic segmentation	ingestion	improved coverage with bounded merge errors	92.73	+4.48	accept
t_5	Option-aware KF (Person-aMem)	filter	cross-benchmark regression signal	−1pp	−1.0	diagnostic flag
t_6	BM25/RRF + Agentic + unified core	retrieval	shared-core gain across diagnostics	93.59	+0.86	accept

The full internal development log contains more than 30 iterations. Table 24 lists only representative rounds with clear evidence artifacts; unlisted rounds are not used as independent empirical claims. Operationally, the timeline alternated among three states: accepted fixes that moved the shared kernel forward, rejected or reverted probes that became negative priors, and pending hypotheses that lacked enough paired evidence for acceptance. This distinction avoids reading the iteration history as monotonic hill-climbing or as evidence that every diagnostic idea worked. A visual timeline is omitted because the discrete three-state model (accept/revert/pending) and the table’s root-cause labels provide an auditable record that a linear diagram would flatten into a misleading monotonic curve.

E.1 Verification Infrastructure

Paired comparison algebra. All acceptance decisions operate on aligned question keys rather than aggregate scores. Given baseline output set Y_{base} and candidate output set Y_{cand} aligned by identifier, the analyzer computes:

$$I = \{i : J_{\text{base}}(i) = 0 \wedge J_{\text{cand}}(i) = 1\} \quad (\text{improved}) \quad (15)$$

$$R = \{i : J_{\text{base}}(i) = 1 \wedge J_{\text{cand}}(i) = 0\} \quad (\text{regressed}) \quad (16)$$

$$B_w = \{i : J_{\text{base}}(i) = 0 \wedge J_{\text{cand}}(i) = 0\} \quad (\text{both-wrong}) \quad (17)$$

$$B_c = \{i : J_{\text{base}}(i) = 1 \wedge J_{\text{cand}}(i) = 1\} \quad (\text{both-correct}) \quad (18)$$

A candidate is accepted only if $|I| > |R|$ and the applicable gate is satisfied. When statistical artifacts are reported, this includes a significance check; otherwise the result remains descriptive.

Statistical gates. The two-stage gate structure is intended to reduce false-positive acceptance while maintaining iteration speed:

- **Stage-1 (probe):** Run on $|S_{\text{probe}}| = 50\text{--}100$ samples. Reject if $\text{Acc}(\theta') < \text{Acc}(\theta_t) - \delta_1$ (typically $\delta_1 = 2\text{pp}$). This filters regressive candidates before full evaluation.
- **Stage-2 (full):** Run on all N samples. When reported, McNemar’s test on discordant pairs ($|I|, |R|$) uses $\alpha = 0.05$, and bootstrap 95% CI on the accuracy difference should exclude zero. Per-category constraint $\forall c : \Delta_c \geq -\epsilon_c$ limits category tradeoffs. If these statistical artifacts are not reported for a result, the result remains descriptive.

Shared-core discipline. The implementation gate requires fixes to be implemented in shared modules (**benchmarks/core/**) whenever possible. This rule is what enabled separate LoCoMo/LongMemEval and PersonaMem branches to later converge into a unified framework without code duplication or test-set-specific shortcuts.

E.2 Detailed Case Studies

Three cases apply the verification infrastructure above to representative D²ACCI iterations.

Case A: Threshold repair via ingestion diagnosis ($t_3 \rightarrow t_4$, +4.48pp). Round t_3 applied deduplication threshold 0.75, which over-merged semantically adjacent but distinct facts. The paired comparison revealed $|R| = 7$ regressions concentrated in the *single-hop factual* category. Diagnostic trace inspection showed a consistent pattern: for 6 of 7

regressed questions, the ground-truth evidence e_i^* was present in the raw dialogue but absent from L0 storage after ingestion—the evidence-preservation ladder classified all 6 as `ingestion_gap`.

Root-cause traces exposed the mechanism: cosine similarity between “prefers Italian food” and “prefers Italian restaurants for dates” was 0.81, exceeding the 0.75 threshold and triggering dedup merge. The merged entry retained only the first variant, losing the date-context distinction that the query required. Similar collapses occurred for temporal updates (“moved to Beijing in 2022” merged with “lives in Beijing”).

Round t_4 addressed this with two changes: (i) raising the dedup threshold to 0.92 to restrict merges to near-duplicate entries; (ii) adding topic-boundary segmentation so that temporally separated mentions of the same entity are treated as distinct episodes. Stage-1 probe on 80 samples showed +3.2pp; the available Stage-2 trace showed a descriptive +4.48pp gain with zero observed per-category regressions. The 7 previously regressed questions all recovered, and 4 additional `ingestion_gap` cases in the *temporal* category were simultaneously resolved.

Case B: Negative result as structured prior (t_5 , rejected). Round t_5 hypothesized that option-aware knowledge filtering would improve PersonaMem MCQ accuracy by injecting retrieved facts matching each answer option. The diagnostic trace from the PersonaMem adapter showed that 12 questions had retrievable option-matching evidence in L0.

Paired comparison revealed an asymmetric outcome: $|I| = 3$ (questions where option-matching evidence disambiguated the correct answer) vs. $|R| = 4$ (questions where injecting distractor-matching evidence confused the model). Trace inspection of the 4 regressions showed a common pattern: the retrieved “evidence” for incorrect options had higher semantic similarity to the query than the correct-answer evidence, causing the generation stage to prefer the wrong option.

The discordant-pair pattern ($|I| = 3, |R| = 4$) did not provide a reliable improvement signal. The change was rejected and archived as a structured negative prior: “*option-aware KF improves precision for unambiguous facts but introduces distractor amplification in borderline preference questions.*” This prior discouraged two subsequent iterations from re-attempting similar filter strategies, avoiding repeated evaluation of the same failure pattern.

Case C: Seesaw detection and conditional narrowing (t_6 sub-iteration). During the development of the Conv3 constraint relaxation (part of the t_6 composite change), an intermediate candidate relaxed the temporal filter threshold from 3 to 5 conversation turns. The diagnostic comparison showed:

- **Improved set** $|I| = 5$: all in *temporal* category, where the relaxation allowed older but relevant evidence to survive filtering.
- **Regressed set** $|R| = 3$: all in *single-hop*, where stale facts from earlier conversations now competed with current facts.

Rather than globally accepting or rejecting, the paired comparison’s per-category breakdown enabled *conditional narrowing*: the relaxation was applied only when the query’s intent classification indicated temporal reasoning (detected via date-entity presence and comparative language markers). The narrowed version achieved $|I| = 5, |R| = 0$ on re-evaluation and passed the configured Stage-2 gate in the available trace. This case illustrates how D²ACCI can transform a category-tradeoff failure into a targeted improvement by leveraging category-level diagnostic granularity.

Predicted failure modes observed. The three limitation categories identified in Section 6.2.1 were each observed during the iteration history:

- **Multi-root-cause coupling** manifested in t_6 : the initial BM25/RRF candidate entangled retrieval improvement with assembly-stage overflow (too many candidates exceeding budget). The 80% coverage requirement forced decomposition into separate retrieval and assembly sub-patches, each independently verifiable.
- **Judge non-determinism** was observed between t_1 and t_2 : a re-run of t_1 ’s accepted configuration showed ± 1.2 pp variance (87.6–89.9%) across three runs. The paired-key comparison eliminated this noise by comparing matched question outcomes rather than aggregate scores.
- **AI iteration without falsifiability** was blocked by design: in two instances, the AI diagnosis agent proposed patches without a human-formulated hypothesis. Both were blocked at Step 1 validation and returned to the human for hypothesis formulation before proceeding.

This “predict-then-observe” pattern suggests that D²ACCI’s failure taxonomy is actionable, supporting earlier architectural checks rather than only post-hoc debugging.

F E²MEND Evolution Artifacts

This section details the E²MEND artifacts behind the bounded-automation claim: authority boundaries, strategy artifact schema, experimental configuration, the three-phase loop, Layer A diagnosis, Gate/Critic rollback rules, candidate traces, and optional search heuristics.

F.1 Authority Boundary and Strategy Space

Table 25 Loop authority.

Decision	Automation Loop: E ² MEND	Governance Loop: D ² ACCI
Search target	Schema-constrained strategy space: prompts, thresholds, routing rules, feature toggles	Framework, protocol, architecture, data schema
Acceptance authority	Can reload only gate-approved strategy artifacts	Approves new capability boundary, benchmark, schema, and gate rules
Reject handling	Revert candidate, update dimension reputation, retry another declared path	Reframe hypothesis, inspect traces, patch framework or protocol if justified
Escalation trigger	Same blocked class repeats or requires a locked-field change	Inner loop reaches its bounded ceiling or new risk appears
Verification	5-Gate/Critic, targeted screen, paired non-regression	Full six-step evidence review plus applicable statistical or descriptive gates
Risk	Low and reversible	Higher, human-reviewed
Goal	Continuous tuning	Extend system capability ceiling

The implemented search space is defined by the same schema-constrained declarative strategy space described in Section 6: a schema-level contract constrains legal fields, value ranges, evolvable dimensions, and cross-field invariants, while strategy artifacts instantiate concrete policies consumed by the runtime. The framework is locked: E²MEND cannot remove the retrieval framework, storage hierarchy, evaluator, or acceptance policy. Candidate mutations are limited to declared strategy paths, including extraction prompts, retrieval thresholds, entity-aware retrieval parameters, rerank limits, source-window toggles, and intent-specific top-*k* multipliers. Prompt changes are permitted only as versioned strategy artifacts and must pass prompt-integrity gates that preserve required output markers. Table 26 provides a schema excerpt listing representative evolvable fields and their guardrails.

Table 26 Strategy guardrails.

Field group	Example fields	Guardrail
Extraction	prompt version, max facts per trajectory, dedup threshold	Required output markers; bounded fact counts
Retrieval	top_k, vector/BM25 weights, entity boost threshold, rerank top_n	Legal numeric ranges; no removal of locked channels
Lifecycle	pruning threshold, decay half-life, compression policy	Cannot delete active correction/forget constraints
Presentation	source-window size, citation mode, output-format prompt	Prompt-integrity and stable-correct replay
Feature toggles	failure correction, conflict handling, conditional triggers	Must declare trigger scope and rollback key

The staged LoCoMo run did not exhaust this combinatorial schema. Candidate policies were selected by diagnosis-driven planning: first prefer dimensions implicated by Layer A traces, then use history stores to avoid reverted directions, and optionally apply UCB1 to balance promising and under-tested paths. The reported +19.16pp is therefore a bounded offline search result, not evidence of optimality over the full schema.

F.2 Experimental Configuration

Table 27 lists the role-separated model stack used in the reported E²MEND run. Internal endpoint URLs are omitted because they are deployment-specific and not part of the algorithmic configuration. The design point is separation of duties: extraction, evolution, judging, and embedding are intentionally assigned to different roles so that strategy search is not evaluated by the same component that proposes it.

This table is included to make the reported run auditable at the role level. It should be read together with the fixed harness and gate criteria, rather than as a recommendation for a particular vendor mix.

Table 27 Model roles.

Role	Model
Fact extraction from conversations	Qwen3.5-9B
Query-intent parsing for retrieval routing	Qwen3.5-9B
Evolution agents (Digester / Planner / Evolver)	claude-opus-4-8
QA evaluation and judging	gpt-4o
Fact indexing embeddings	azure_openai/text-embedding-3-small

This configuration fixes the reported run context for reproducibility; it does not imply that the same roles require these exact vendors or endpoints.

F.3 Three-Phase Evolution Loop

Algorithm 2 expands the Observe–Improve–Verify loop summarized in the main text. It maps each reported E²MEND candidate to the point where evidence is gathered, a strategy mutation is proposed, and acceptance gates decide whether the serving strategy may change.

The pseudocode is intentionally implementation-neutral: names such as PLAN, MUTATE, and FIVEGATE denote artifact-producing steps, not required class or function names.

Algorithm 2: E²MEND strategy loop.

Input: Current strategy s_t , benchmark set $\mathcal{B}$, YAML schema Γ , history stores $\mathcal{H}$

Output: Accepted, pending, reverted, or restored strategy s_{t+1}

```

1 Phase 1---Observe:
2 Evaluate  $s_t$  on  $\mathcal{B}$  to obtain score report  $r_t$ 
3 Run Layer A deterministic diagnosis over  $r_t$ : compute extraction_coverage and retrieval_rank,
   then assign root-cause stage  $\sigma_t$ 
4 Compress Layer A traces into actionable digest  $g_t$ 

5 Phase 2---Improve:
6 Select top-impact mutable dimensions  $D_t \leftarrow \text{PLAN}(g_t, \mathcal{H})$ 
7 Generate candidate strategy set  $\mathcal{C}_t \leftarrow \text{MUTATE}(s_t, D_t, \Gamma)$  through bounded YAML mutation

8 Phase 3---Verify:
9 foreach  $c \in \mathcal{C}_t$  do
10   if  $c$  violates declared strategy scope then continue
11   if FIVEGATE( $c, \Gamma, \mathcal{B}, \mathcal{H}$ ) rejects then continue
12   if CRITIC( $c, \mathcal{H}$ ) rejects then continue
13   Run targeted screen on failure and stable-correct subsets
14   if screen shows no positive paired delta then continue
15   if full-evaluation budget is unavailable then
16     Record  $c$  as a pending-champion candidate for next-round confirmation
17   continue
18 end
19 Run full paired item-level comparison between  $c$  and  $s_t$ 
20 if configured acceptance and bounded regression checks pass then
21   Persist decision reports, digests, and cross-round stores into  $\mathcal{H}$ 
22   return  $s_{t+1} \leftarrow c$ 
23 end
24 end
25 Log rejection outcome, update dimension reputation in  $\mathcal{H}$ , and persist the best pending champion if any
26  $s_{t+1} \leftarrow s_t$ 
27 if  $\text{Acc}(s^*) - \text{Acc}(s_{t+1}) > \tau_{\text{rollback}}$  then  $s_{t+1} \leftarrow s^*$ 
28 return  $s_{t+1}$ 

```

The loop makes the candidate trace interpretable: each row records a mutation that survived or failed the Verify phase. For the staged E²MEND run reported here, acceptance used configured offline checkpoint gates under the same fixed model and judge configuration across stages. Because p-values, confidence intervals, and repeated-run variance are not reported for the selected rows, the accepted checkpoints summarized in Table 10 and the candidate outcomes in Table 29 are treated as descriptive full-benchmark evidence, not statistical significance claims.

F.4 Layer A Diagnostic Decision Tree

Algorithm 3 expands the Layer A deterministic diagnosis referenced in the main E²MEND loop (line 3 of Algorithm 2). For the LoCoMo setting used here, each wrong-answer sample is assumed to have complete evidence annotations. Layer A first checks whether the annotated source evidence was ingested into memory, then whether that evidence-derived memory was retrieved, and finally uses a single LLM classification call to distinguish generation failure from lossy ingestion.

Algorithm 3: Layer A diagnosis.

Sample: Wrong-answer LoCoMo sample $s = (q, a^*, E)$

Context: Provider P ; memory store $\mathcal{F}$; complete evidence ids E

Output: Root-cause label $\ell \in \{\text{generation_error}, \text{ingestion_gap}, \text{retrieval_gap}, \text{uncertain}\}$

```

1 Phase 1: evidence-to-memory coverage
2  $(D_E, W_E) \leftarrow$  dialogue ids and ingestion-window ids implied by  $E$ 
3  $\mathcal{F}_E \leftarrow \text{EvidenceFacts}(D_E, W_E, \mathcal{F})$  // source ids overlap gold evidence
4 if  $\mathcal{F}_E = \emptyset$  then return ingestion_gap

5 Phase 2: retrieval coverage
6  $(I, X) \leftarrow \text{Retrieve}(P, q)$  // retrieved ids and texts
7  $r \leftarrow \text{Rank}(\mathcal{F}_E, I, X)$  // best 1-based rank; -1 if absent
8 if  $r < 0$  then return retrieval_gap

9 Phase 3: source/fact faithfulness judgment
10  $u \leftarrow \text{SourceWindow}(I_r)$  // original source window
11  $c \leftarrow \text{LLM-Classify}(q, a^*, u, X_r)$  // Full_Coverage / Source_Only / Uncertain
12 if  $c = \text{Full\_Coverage}$  then return generation_error
13 if  $c = \text{Source\_Only}$  then return ingestion_gap
14 return uncertain

// EvidenceFacts and Rank are deterministic; only Phase 3 uses an LLM.

```

Note. This decision tree describes the evidence-complete LoCoMo path used in the reported analysis. The implementation also contains fallback checks for missing or unresolved evidence ids, but those branches are omitted here because they are not active under this setting. In deployment-like settings without complete annotations, Layer A should be treated as heuristic and audit-assisted, not a deterministic earliest-loss claim. Because facts are extracted from ingestion windows rather than individual dialogue turns, **ingestion_gap denotes missing evidence linkage at the dialogue/window-id level**. A semantically equivalent fact without source overlap is therefore treated as unresolved evidence alignment, **not as evidence that no equivalent memory exists**.

Notation. For the current sample, D_E and W_E are the dialogue ids and ingestion-window ids derived from its gold evidence annotation; $\mathcal{F}_E \subseteq \mathcal{F}$ is the subset of stored memories whose source ids overlap that sample-specific evidence scope. I and X are the retrieved memory ids and texts; r is the first retrieved rank of any evidence-derived memory. With complete LoCoMo evidence, no $\mathcal{F}_E$ means ingestion failed, while no retrieved rank for $\mathcal{F}_E$ means retrieval failed. LLM-CLASSIFY compares the question, gold answer, original source window u , and retrieved fact X_r . **Full_Coverage** means the fact contains the answer but the agent failed to use it; **Source_Only** means the source contains the answer but extraction lost or weakened it.

F.5 Gate, Critic, and Rollback Rules

The rollback rule is applied after candidate rejection or acceptance decisions: if the current served strategy falls more than τ_{rollback} below the best-ever accepted checkpoint, the loop restores the persisted best strategy. The Gate pipeline executes in fail-fast order, then delegates qualitative risk assessment to an independent Critic. Table 28 states the rule levels used in the reported run.

Table 28 Gate, Critic, and rollback rules.

Check	Rejects if	Effect on search
Structure gate	Mutation path is absent from schema or touches locked framework/protocol fields	Candidate is discarded; issue returned to D ² ACCI if capability change is needed
Novelty gate	Same path/direction was already reverted or marked exhausted	Planner lowers reputation and explores a different dimension
Range gate	Value leaves legal bounds (e.g., <code>top_k</code> , score thresholds, max facts)	Candidate is canonicalized if still within bounds, otherwise rejected
Prompt-integrity gate	Required JSON markers, variables, or safety/citation clauses are removed	Candidate is rejected before evaluation
Replay gate	Stable-correct probe regresses beyond configured tolerance	Candidate is rejected; regressed ids are written to history
Full paired acceptance check	Full evaluation lacks positive paired delta, has excessive regressions, or fails the configured McNemar/practical threshold	Candidate is rejected, retained as pending only when promising, or accepted into the served strategy
Critic soft review	Dimension has weak reputation, broad affected scope, duplicate direction, or limited actionability	Adds warnings or blocks expensive evaluation when risk is high
Rollback safeguard	Current served strategy falls more than τ_{rollback} below the best-ever accepted checkpoint	Restores the persisted best strategy

The Critic prompt is narrow by design: it receives the candidate diff, affected schema path, recent dimension reputation, probe deltas, and examples of improved/regressed questions, then labels the mutation as actionable, risky, duplicate, or out-of-scope. It cannot override hard gates. If a candidate is rejected, E²MEND writes the reason to `candidate_outcomes` and updates `dimension_effects`; repeated rejection reduces the Planner score for that dimension. If the same class of candidate is blocked three times because it requires framework, schema, or protocol changes, the loop escalates the issue to the human-governed D²ACCI backlog rather than continuing autonomous search.

F.6 Candidate Trace and Search Heuristics

Candidate trace. Table 29 is an audit trace for the staged E²MEND run, not an implementation manual. The candidate id column records the corrected evolution-step numbering, and the strategy dimension column is a stable artifact namespace rather than an executable code path. The table records the evidence trail behind the reported gain: each row summarizes one bounded strategy change or grouped checkpoint, evaluated under the locked LoCoMo harness and then accepted, rejected, or marked no-effect according to gate evidence.

The trace shows three patterns. First, gains are not monotonic: self-correction and broad raw-dialogue fallback were rejected because they introduced regressions or unstable evidence. Second, the largest accepted gains came from evidence admission, typed-state abstraction, and verified candidate adoption, suggesting that E²MEND primarily improved the memory substrate rather than final-answer phrasing. Third, later gains became smaller but lower-risk, with deterministic support clustering and evidence-supported consolidation adding accuracy without observed regressions.

Optional UCB1 hypothesis tree. When `features.hypothesis_tree` is enabled, Planner exploit targets and retrieval-diagnostic seeds are materialized as hypothesis nodes keyed by strategy path and direction, e.g., `retrieval.top_k::increase`. Each node stores visits, total reward, status (`active`, `confirmed`, `refuted`, `exhausted`), best observed delta, and evidence for/against the direction.

Selection follows UCB1 over active nodes:

$$\text{UCB}(h) = \bar{r}_h + c(a) \sqrt{\frac{\log N}{n_h}}, \quad (19)$$

where $\bar{r}_h$ is the normalized historical reward of hypothesis h , n_h is its visit count, N is the parent visit count, and $c(a)$ is an exploration constant coupled to Planner actionability a . Low actionability increases $c(a)$ and allows broader exploration; high actionability reduces $c(a)$ and favors historically productive directions. After candidate verification, the observed paired delta is written back to the node so repeated positive, negative, or no-effect trials can confirm, refute, or exhaust a direction.

Table 29 Representative candidate outcomes from staged E²MEND.

ID(s)	Strategy dimension	Candidate change	Decision	Acc.	Δ	Gate evidence
000	Initial memory strategy	Baseline strategy before E ² MEND evolution	baseline	0.756	–	1164/1540
005	Semantic multi-hop reranking	Enable graph-based reranking for semantic multi-hop retrieval	no-effect	–	≈ 0.000	no target-set improvement
006	Answer verification	Add self-correction before final answer	rejected	0.798	–0.018	+4 / –6 screen examples
027	Evidence-grounded context	Attach local dialogue evidence into answer context	accepted	0.818	+0.062	+96 / no recorded regression
033–034	Scoped evidence gate	Restrict candidate use to evidence-supported cases	accepted	0.845	+0.089	+42 / limited regressions
035–036	Raw-dialogue fallback	Broaden fallback search over raw dialogue	rejected	–	unstable	+10 / –7 screen examples
037–047	Typed state abstraction	Convert evidence into typed temporal, location, plan, preference, slot, and ledger state	accepted	0.871	+0.115	+177 / no recorded regression
048–060	Verified candidate bank	Adopt only candidates that pass support and non-regression checks	accepted	0.915	+0.159	+245 / no recorded regression
064	Support clustering	Use deterministic support clusters instead of selector choice	accepted	0.937	+0.179	+34 / no regressions
067	Evidence-supported consolidation	Consolidate candidate bank using explicit evidence support	accepted	0.947	+0.192	+16 / no regressions

G Extended LiteMem Artifacts

This section documents LiteMem engineering details: repository schema, directory hierarchy, Git-based provenance, ranking/decay equations, lifecycle pseudocode, and the explicit scaling boundary. Together, Tables 30 and 31 explain the per-file metadata that makes a plain Markdown repository directly rankable and indicate where those files live at runtime.

Table 30 Frontmatter schema.

Field	Purpose	Runtime Usage
id, type, status	stable identity and lifecycle state	routing, active/deprecated/archived filtering
title, summary, keywords	retrieval surface	grep/TF-IDF-like scoring and summary selection
importance, last_accessed_at	salience model	exponential decay ranking
access_count, skip_count	feedback statistics	decay reset and organization diagnostics
source, supersedes	provenance and update lineage	traceability, conflict resolution, rollback
token_estimate, confidence	packing and reliability control	budget-aware context assembly

Table 31 Repository hierarchy.

Directory	Semantic Role	Use Pattern
user/style.md	response style and interaction constraints	injected every round or kept at highest priority
user/profile.md	long-term user or speaker profile	searched and often prepended for personalization
sessions/	episodic conversation memories	keyword searched, ranked, full-read on demand
entity/	entity/topic aggregated memories	consolidation output and multi-hop support
knowledge/skill/	repository-local procedures or task routines	invoked when the task scenario matches
knowledge/learning/	corrections and learned rules	high-confidence factual correction source
daily/	raw conversation logs	fallback provenance and missed-write recovery source
_index.md	global memory map	fast scan before deep retrieval

The frontmatter is operational metadata: each field affects retrieval, context packing, provenance, or lifecycle filtering. This hierarchy separates globally relevant user constraints from episodic logs, consolidated entity memories, and learned procedures, which is why LiteMem can retrieve selectively without injecting the whole repository. The listing provides a concrete instantiation with the running example:

```

litemem/ -- repository layout (training-handoff scenario)

litemem/
  _index.md          # global map and active-memory summary
  user/
    profile.md       # L2-like long-term profile
    style.md         # response constraints and preferences
  sessions/
    2026-07-10_training_handoff.md # SM/L1-like episodic summary
  entity/
    ethan.md         # consolidated entity memory
  knowledge/
    skill/equipment_check.md # PCSM-like procedure
    learning/corrections.md # user corrections and invalidations
  daily/
    2026-07-10.md    # append-only raw write-ahead log

--- run diagnostics ---
git log -- daily/2026-07-10.md
git diff HEAD~1 -- user/profile.md sessions/2026-07-10_training_handoff.md

--- router-policy (lexical recall) ---
grep -R "training bag|jersey|spare shoes" user sessions entity daily

```

In this layout, one memory is usually one Markdown file or one section inside a consolidated entity file; one Git commit records a coherent capture/organization action rather than a single fact. Diagnostics use standard repository tools: `git log` records when an item appeared or was superseded, `git diff` identifies which fields changed, and `grep/ripgrep` traces whether a lexical recall failure is due to missing content or ranking/selection.

Scaling boundary. A synthetic file-count table is omitted because the current evaluation did not isolate repository size as an independent variable. The planned scaling experiment will vary file count, average file length, index freshness, and query type, then report retrieval latency, selected-evidence recall, and answer accuracy. Until that experiment is run, statements about “hundreds of files” are engineering guidance from prototype use, not a measured performance curve.

Ranking and decay equations. LiteMem ranks candidates with lexical relevance, stored importance, recency, and explicit access feedback:

$$\text{Score}(m, q_t, t) = \lambda_1 \text{Lex}(m, q_t) + \lambda_2 \text{Imp}_t(m) + \lambda_3 \exp(-\Delta t / \tau) + \lambda_4 \text{Access}(m). \quad (20)$$

In these equations, τ and τ_m are half-life-like timescale parameters used by the repository-native implementation, not benchmark-wide constants. The reported LiteMem runs use configuration-level values selected for the simulated LoCoMo timeline and keep them fixed within a run; sensitivity to these local decay settings is treated as an implementation issue, not a main-text claim. Idle organization uses decayed importance:

$$\text{Imp}_t(m) = \text{Imp}_0(m) \cdot \exp(-\Delta t_m / \tau_m) + \eta \cdot \text{access_count}(m) - \rho \cdot \text{skip_count}(m). \quad (21)$$

Algorithm 4 ties the schema and hierarchy back to execution: retrieval reads ranked Markdown files, capture writes new memories with metadata, and organization uses Git history plus decay signals to keep the repository auditable.

H Evaluation Protocol Details

This section collects evaluation protocol details: claim-check matrix (Table 32), root-cause distribution across D²ACCI iterations (Table 33), and resource accounting (Table 34). It also marks which experiments remain pending rather than inferred.

Algorithm 4: Repository lifecycle.

Input: Conversation turn or session x , memory repository $\mathcal{R}$, query/task q_t , turn t **Output:** Updated repository $\mathcal{R}'$ and retrieved context C

- 1 **Recall before answer:**
 - 2 Extract keywords $K(q_t)$; expand synonyms/entities; grep Markdown files in `user/knowledge/entity/sessions/daily`
 - 3 Rank candidates by Eq. 20; always prepend active `user/profile.md` and `user/style.md`
 - 4 If LLM-assisted mode is enabled, present title+summary table and select candidate IDs
 - 5 Lazy-load selected full text; for `daily/` logs, extract only lines around keyword matches
 - 6 Increment `access_count`; update `last_accessed_at`
 - 7 **Capture after interaction:**
 - 8 Append raw conversation to `daily/YYYY-MM-DD.md` as a write-ahead log
 - 9 If structured write is enabled, create or update memory file with YAML frontmatter and `SUMMARY_END` marker
 - 10 **Organization during idle/session boundary:**
 - 11 Compute decayed importance by Eq. 21; archive memories below threshold
 - 12 Run LLM consolidation over memory summaries and recent daily logs
 - 13 Execute merge/deprecate/update-profile actions; rebuild `_index.md` atomically
 - 14 If Git is enabled, commit memory diff for audit and rollback
-

Table 32 Claim checks.

Claim	Operational definition	Current status	Evidence / artifact	Falsified or weakened if...
Long-context QA	Retrieve grounded evidence over long histories	Controlled-reference reported	Public benchmark traces; unified-core harness	Shared-harness reruns lose parity or evidence traces show answer correctness without retrieved support
Bounded strategy evolution	Tune policy without editing locked framework code	Locked offline benchmark reported	Strategy artifacts, candidate outcomes, rollback records	Accepted candidates repeatedly regress categories or require hidden framework changes
Personalized procedures (PCSM)	Store user-specific conversational procedures beyond facts	Design-only; benchmark pending	PCSM schema and trigger-procedure contracts	A standardized PCSM benchmark or ablation shows no benefit over factual preference memory
Cross-device causality	Fuse asynchronous events into causal evidence	Preliminary/internal; external validation pending	Internal MemFuseBench, FusionSession graphs, provenance traces	Removing causal/BELONG edges causes negligible drop, external validation fails, or conflict arbitration remains below baselines
Multimodal grounding	Convert visual observations into structured memory evidence	Module-level reported; IKB ablation not reported in main	Mem-Gallery evaluation, IKB traces, error taxonomy	IKB ablation shows no gain over standard VLM/RAG retrieval on the same 1,711-Q harness
Edge deployment	Run memory without vector-database infrastructure	Transfer run reported; scaling pending	Markdown/Git repo, file paths, diffs, LoCoMo-aligned transfer run	File-native run loses most of the service-side improvement or scaling tests miss the latency target at the target size

D²ACCI root-cause summary. Table 33 records a representative root-cause distribution used to interpret the D²ACCI iteration trace. It is kept in the appendix because the primary claim is the direction of diagnostic shift, not the full per-round ledger. The table compares failure composition across iterations rather than serving as an independent benchmark score; the trend indicates ingestion gaps shrinking while retrieval and generation become the dominant remaining surfaces.

Table 33 D²ACCI root causes.

Root Cause	Round 1	Phase 1	Phase3_fix2	unified-v3
retrieval_gap	60%	72%	68%	71%
ingestion_gap	30%	15%	5%	2%
generation_error	8%	12%	25%	26%
judge_ambiguous	2%	1%	2%	1%

Resource accounting. The cost table reports the earlier human-driven D²ACCI phase, where evaluation used gpt-4.1-mini. It is not the resource profile of the reported E²MEND run. Table 34 is included to separate historical human-driven iteration cost from the automated E²MEND evidence reported in the main text.

Table 34 D²ACCI cost.

Resource	Details	Estimate
Human engineer	2–4 h/round × 30 rounds	~100 h
LLM (coding)	~3,000 Claude Code messages	\$60–120
LLM (eval)	gpt-4.1-mini, \$4–6/round	\$200–300
Wall-clock	Stage 1: 30 min; Stage 2: 6 h	~200 h